

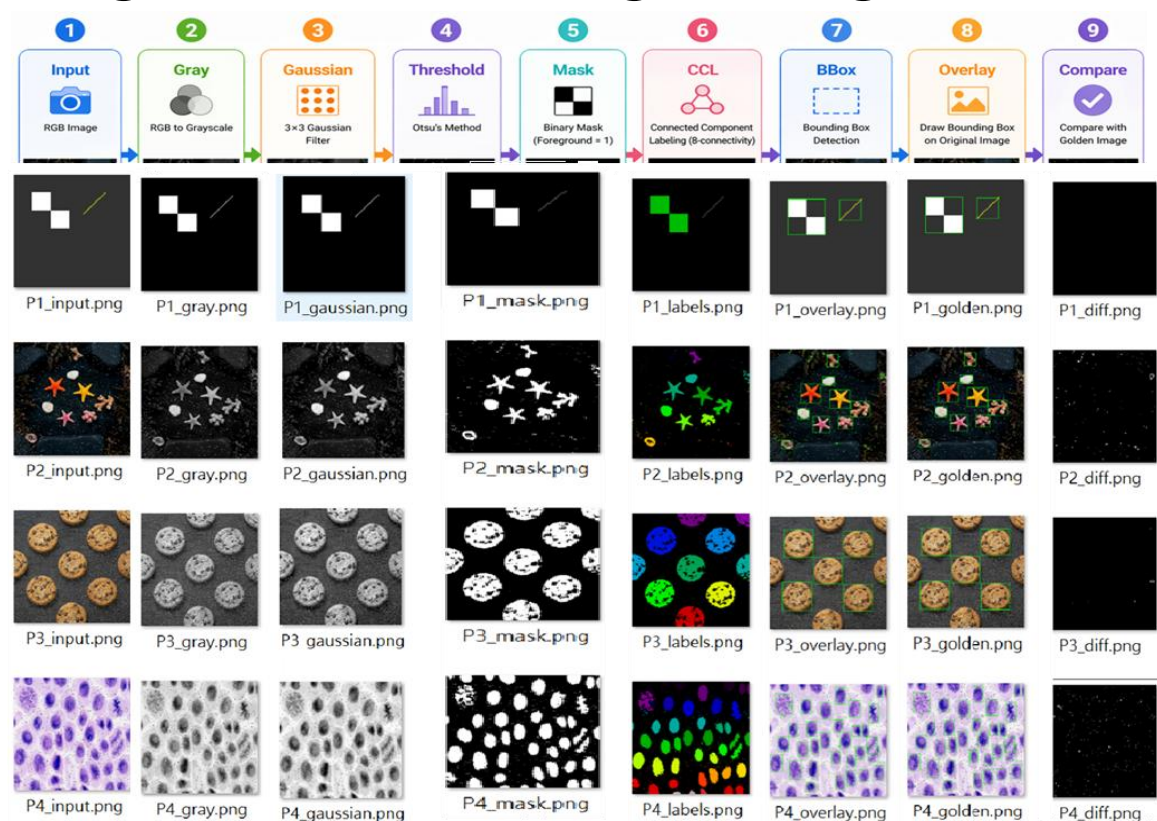
National Cheng Kung University

Department of Electrical Engineering

Introduction to VLSI CAD (Spring 2026)

Lab Session 7

Design of Draw Bounding Box Engine



Name	Student ID	
彭靖凱	E24134778	
Practical	Points	Marks
Lab 7	100	
Notes		

Summary

Hardware			
		RTL All Pass(~/X)	SYN All Pass(~/X)
Bicubic Resize		~	
		RTL Number of Errors	Synthesis Number of Errors
Pattern 1		0	0
Pattern 2		0	0
Pattern 3		0	0
Pattern 4		0	0
Grade Class			
Total Number of Errors		Class (A/B/C/D/E)	
0		A	
Synthesis result			
Area	P4 Simulation time (ps)		Area * P4 Simulation time (ps)
376225.33915	5997155029		2.25628168*10 ¹⁵
Superlint(number of inline messages)			
Total lines	Warning	Error	coverage(%)
944	92	0	90.25% (≥90%)

Note: You must complete and fill out this form with your design information!!!

All P1~P4 patterns passed with 0 errors in both RTL and synthesis simulation.
Pixel-perfect implementation achieved.

Deliverables

- 1) All Verilog codes including testbenches, .bmp and .txt should be uploaded.
- 2) NOTE: Please **DO NOT** include source code in the paper report!
- 3) NOTE: Please **DO NOT** upload waveforms (.fsdb or .vcd)!
- 4) If you upload a dead body which we can't even compile, you will get NO credit!
- 5) All Verilog file should get at least **90%** SuperLint Coverage.
- 6) All homework requirements should be uploaded in this file hierarchy, or you will not get full credit. If you want to use some sub modules in your design but you do not include them in your tar file, you will get 0 point.



Fig.1 File hierarchy for Homework submission

Lab 7: Draw Bounding Box Engine

The design inside the bicubic block can be completed by your free will, but do not modify the I/O ports of the bicubic block. The block diagram of the testbed-DUT (design under test) system is as shown in Fig2.

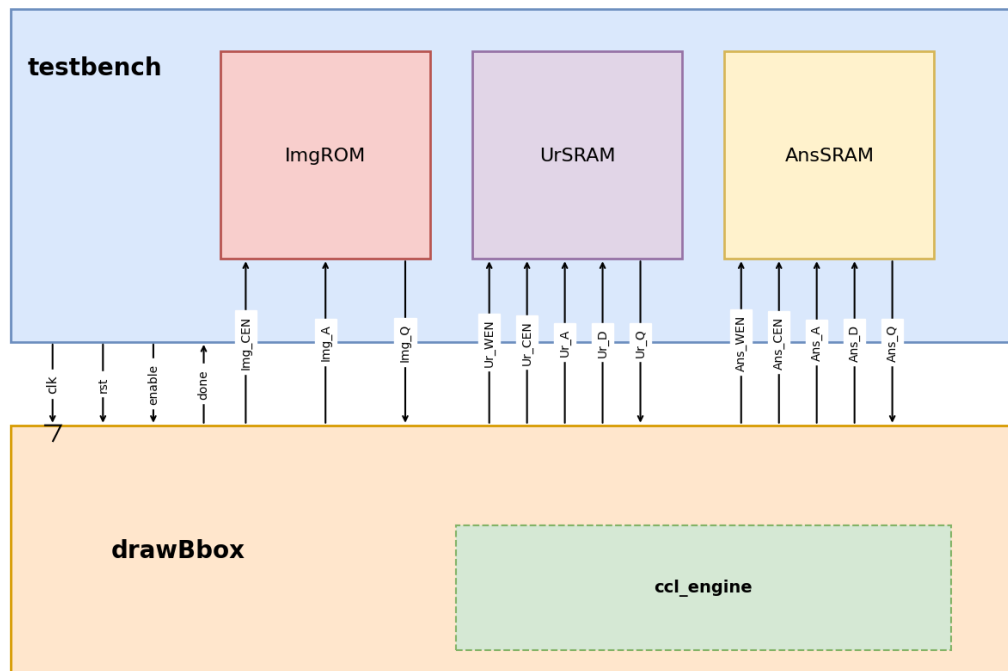


Fig2. The block diagram of Draw Bounding Box circuit

圖 2 為 Draw Bounding Box Engine 的 testbed-DUT 系統架構圖。testbench 負責提供 clk、rst、enable 訊號，並建立 ImgROM、UrSRAM 與 AnsSRAM 等記憶體模型。drawBbox 為本作業的主要設計模組，負責控制所有記憶體讀寫訊號，包含 Img_CEN、Img_A、Ans_CEN、Ans_WEN、Ans_A、Ans_D、Ur_CEN、Ur_WEN、Ur_A 與 Ur_D。

系統運作時，drawBbox 會先由 ImgROM 讀取 32-bit RGB pixel，接著依序執行 RGB 轉灰階、3×3 Gaussian filter、Otsu threshold、binary mask、auto-polarity correction、8-connected CCL、bounding box extraction 與 green overlay。UrSRAM 可作為中間資料或 CCL 標記資訊的暫存空間，AnsSRAM 則用來儲存最終畫好 bounding box 的輸出影像。

其中 ccl_engine 是整個系統中負責 connected component labeling 與 bounding box 座標擷取的核心單元。它會將前景像素分群，找出每個 connected component 的 Xmin、Xmax、Ymin、Ymax，並排除接觸邊界的物件。當所有 pixel 處理完成後，drawBbox 會將 done 訊號拉高，通知 testbench 可以開始比對輸出結果與 golden data。

根據本次實作的 drawBbox.v, drawBbox 模組為整個系統的最高層硬體引擎，負責從 ImgROM 讀取原始 RGB 影像資料，透過 UrSRAM 儲存中間處理資料，並將最終繪製完成的影像寫入 AnsSRAM。本設計是在既有記憶體與控制訊號下完成所有影像處理流程。

系統內部由有限狀態機控制。電路啟動後，會先初始化內部暫存器、計數器與記憶體控制訊號，接著依序讀取 256×256 RGB 影像。之後系統會執行灰階轉換、Gaussian 濾波、Otsu 門檻計算、binary mask 產生、自動極性修正、連通元件標記、bounding box 座標擷取，以及綠色邊界框繪製。最後將處理後的影像寫入 AnsSRAM，並將 done 訊號拉高，通知 testbench 開始進行 golden data 比對。

圖中的 ccl_engine 代表 connected component labeling 與 bounding box extraction 的核心單元。此單元會根據 binary mask 將前景像素分群，合併等價 label，並記錄每一個 connected component 的最小與最大 x/y 座標。若物件接觸影像邊界，則依照 Lab7 規定不繪製 bounding box；若為有效物件，則使用 32'h0000_FF00 作為綠色邊界框顏色。

➤ **Port list of Draw Bounding Box Engine:**

Signal	Type	Bits	Description
<u>clk</u>	input	1	Positive-edged triggered
<u>rst</u>	input	1	active high reset signal
<u>done</u>	output	1	Active high finish signal
<u>enable</u>	input	1	Active high enable signal to start processing
<u>Img_CEN</u>	output	1	Active low read enable signal for <u>ImgROM</u>
<u>Img_A</u>	output	16	16 bits address for <u>ImgROM</u>
<u>Img_Q</u>	input	32	Read 32 bits unsigned data from <u>ImgROM</u>
<u>Ans_WEN/Ur_WEN</u>	output	1	(Low write/high read) enable signal for <u>AnsSRAM/ UrSRAM</u>
<u>Ans_A/ Ur_A</u>	output	16	16 bits address for <u>AnsSRAM/ UrSRAM</u>
<u>Ans_D/ Ur_D</u>	output	32	Write 32 bits unsigned data to <u>AnsSRAM/ UrSRAM</u>
<u>Ans_Q/ Ur_Q</u>	input	32	Read 32 bits unsigned data from <u>AnsSRAM/ UrSRAM</u>
<u>Ans_CEN/ Ur_CEN</u>	output	1	Active low chip enable signal for <u>AnsSRAM/ UrSRAM</u>

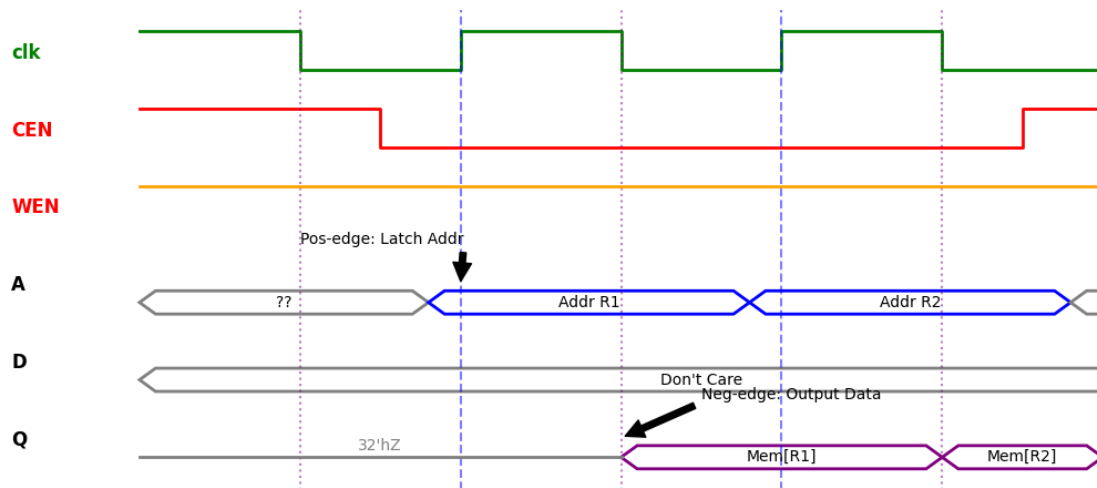


Fig3. example waveform for RAM read operation

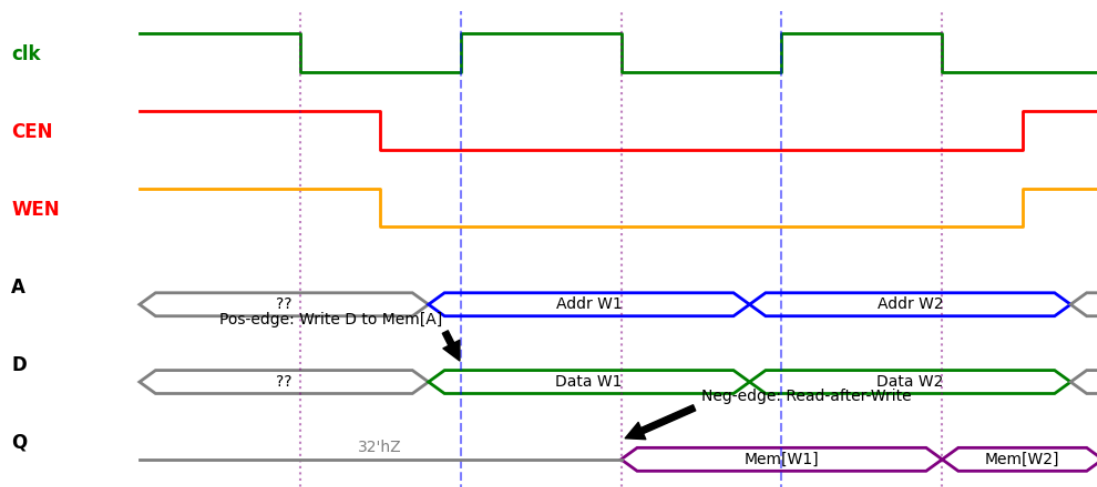


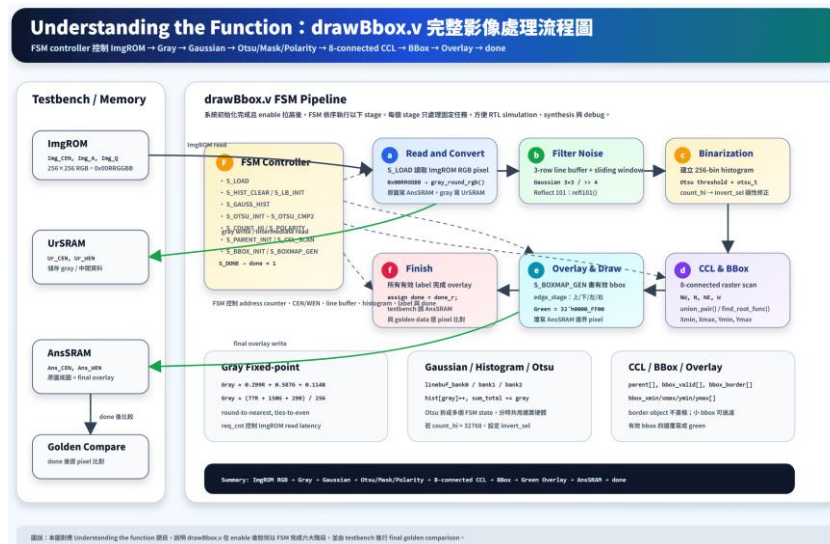
Fig4. example waveform for RAM write operation

➤ Understanding the function: (for example)

Once system is initialized, it

- Read and Convert:** Reads the original RGB pixels from the testbench (ImgROM), converts them into grayscale format, and stores the results into the working memory (AnsSRAM).
- Filter Noise:** Applies a 3x3 Gaussian blur filter to the grayscale image using a sliding window. This smooths out high-frequency noise and overwrites the memory with the blurred image.
- Binarization:** Binarizes the image into a 0/1 mask (foreground vs. background) based on the calculated Otsu threshold, and writes this mask back to the memory.
- CCL & BBox Extraction:** Activates the 8-connected CCL (Connected-Component Labeling) engine. The engine identifies connected object pixels and extracts their bounding box coordinates (X_min, X_max, Y_min, Y_max), storing these limits into the tracking memory (UrSRAM).
- Overlay & Draw:** Copies the original RGB image into the final output memory. It then iterates through the extracted bounding box coordinates to draw 1-pixel green boundary lines around each valid detected object.
- Finish:** Asserts the done signal to notify the testbench that the final annotated image is ready.

A: 當系統初始化完成並且 enable 訊號被拉高後，本設計的 drawBbox.v 會由 FSM controller 依序執行完整的 Draw Bounding Box 影像處理流程。整體流程包含原始 RGB 影像讀取、灰階轉換、Gaussian filter、Otsu threshold、binary mask 與 auto-polarity correction、8-connected CCL、bounding box extraction、green overlay，以及最後輸出完成訊號 done。



a) Read and Convert

系統首先進入 S_LOAD 階段，透過 Img_CEN 與 Img_A 從 ImgROM 讀取原始 256×256 RGB image。每一個 pixel 為 32-bit 資料，格式可視為 0x00RRGGBB。由於 ImgROM read data 有讀取延遲，因此程式中使用 req_cnt 控制讀取位址與資料寫入時序。

讀取到的原始 RGB pixel 會同時進行兩件事。第一，原始 RGB pixel 會寫入 AnsSRAM，作為最後 green bounding box overlay 的底圖。第二，RGB pixel 會透過 gray_round_rgb() function 轉換成灰階值，並寫入 UrSRAM 作為後續 Gaussian filter、Otsu threshold 與 CCL 的中間資料。

灰階轉換使用固定點近似計算：

$$\text{Gray} \approx 0.299R + 0.587G + 0.114B$$

在 RTL 中則以整數權重實作：

$$\text{Gray} \approx (77R + 150G + 29B) / 256$$

並且加入 round-to-nearest, ties-to-even 的 rounding 判斷，使灰階結果更接近 Lab7 規格要求。

b) Filter Noise

完成灰階資料儲存後，FSM 進入 histogram 清除與 line buffer 初始化階段。程式先在 S_HIST_CLEAR 中清除 hist[0] 到 hist[255]，接著在 S_LB_INIT 中從 UrSRAM 讀取灰階資料並建立初始 3-row line buffer。

本設計沒有使用完整的 gray_mem[65536] register array，而是將灰階資料存放在 UrSRAM，再使用三個 256-byte line buffer bank：

linebuf_bank0

linebuf_bank1

linebuf_bank2

並透過 lb_prev_sel、lb_curr_sel、lb_next_sel 進行 bank selector swap。這樣可以避免使用過大的 full-image register array，也能降低 synthesis 時的硬體複雜度。

Gaussian filter 在 S_GAUSS_HIST 階段執行。系統使用 gaussian_lb() function 讀取 line buffer 中的 3×3 window，並套用 Gaussian kernel：

1 2 1

2 4 2

1 2 1

因為權重總和為 16，所以 RTL 中可使用 right shift 取代除法，降低硬體成本。

邊界處理則使用 refl101() function 實作 Reflect 101 padding，使影像邊界的濾波結果符合 Lab7 的要求。

c) Binarization

在 Gaussian filter 的同時，系統會建立 256-bin histogram。每計算出一個 Gaussian gray value，就將對應的 hist[gray] 加一，並累加 sum_total。完成整張影像的 histogram 後，FSM 進入 Otsu threshold 計算階段。

Otsu threshold 並不是在單一 combinational block 中完成，而是拆成多個 FSM state，例如：

S_OTSU_INIT

S_OTSU_SCAN

S_OTSU_LHS

S_OTSU_DENOM

S_OTSU_SQUARE

S_OTSU_CMP1

S_OTSU_CMP2

這樣的設計可以讓 Otsu 計算中的乘法器資源分時共用，避免一次產生過多大型乘法器，降低 Design Compiler 合成負擔。完成 Otsu 計算後，系統會得到最佳 threshold otsu_t。

接著，系統在 S_COUNT_HI 與 S_POLARITY 階段進行 auto-polarity correction。程式會統計高於 threshold 的 pixel 數量 count_hi。若高亮 pixel 數量超過半張影像，也就是大於 32768，代表可能把背景誤判為前景，因此設定 invert_sel，在後續 CCL 判斷 foreground 時進行反相修正。

d) CCL & BBox Extraction

完成 threshold 與 polarity 判斷後，系統進入 CCL 相關階段。首先在 S_PARENT_INIT 中清除 parent[]、bbox_valid[]、bbox_border[]、bbox_xmin[]、bbox_xmax[]、bbox_ymin[] 與 bbox_ymax[] 等 CCL 與 bounding box 相關陣列。接著在 S_LB_INIT2 中重新載入 line buffer，作為 CCL 掃描時即時計算 Gaussian 與 foreground 的基礎。

CCL 主體在 S_CCL_SCAN 中執行。本設計使用 8-connected CCL，採用 raster scan 的方式由左到右、由上到下掃描整張影像。對於每一個 foreground pixel，系統會檢查已經掃描過的四個鄰居：

NW, N, NE, W

在程式中分別對應到：

nw_label_hold

row_label[ccl_x]

row_label[ccl_x + 1]

left_label

若四個鄰居都沒有有效 label，系統會分配新的 label；若只有一個鄰居有 label，則沿用該 label；若多個鄰居具有不同 label，則選擇其中一個 label，並透過 union_pair() 建立 label equivalence。後續再使用 find_root_func() 找出 root label，使同一個 connected component 的不同 provisional labels 可以合併為同一物件。

在 CCL 掃描過程中，系統也會同步更新 bounding box 資訊。對於每個有效 label，系統會記錄其最小與最大座標：

Xmin, Xmax, Ymin, Ymax

若物件碰到影像邊界，則會設定 bbox_border[label]，後續不會對該物件繪製 bounding box。完成 CCL 掃描後，FSM 進入 S_BBOX_INIT，將等價 label 的 bounding box 合併到 root label，確保最後每個 connected component 都有正確的 bounding box 座標。

e) Overlay & Draw

完成 CCL 與 bounding box extraction 後，FSM 進入 S_BOXMAP_GEN 階段，開始將有效 bounding box 畫回 AnsSRAM 中的原始 RGB 影像。系統會逐一

檢查每個 label，如果該 label 無效、碰到邊界，或 bounding box 尺寸太小，則跳過不畫框。

對於有效的 bounding box，系統使用 edge_stage 與 edge_ctr 分階段繪製四條邊：

上邊界

下邊界

左邊界

右邊界

繪製時，系統將 AnsSRAM 的 write enable 打開，並將對應 pixel 寫成 Lab7 指定的綠色常數：

32'h0000_FF00

由於 AnsSRAM 在 S_LOAD 階段已經先存入原始 RGB image，因此 overlay 階段只需要將 bounding box 邊界位置覆寫成綠色即可，其餘 pixel 仍保留原圖內容。

f) **Finish**

當所有有效 label 都完成 bounding box overlay 後，FSM 進入 S_DONE。在此狀態中，done_r 會被拉高，並透過：

assign done = done_r;

輸出給 testbench。當 testbench 偵測到 done 之後，會讀取 AnsSRAM 中的最終影像，並與 golden data 進行逐 pixel comparison。若所有 pixel 均一致，terminal 會顯示 Simulation PASS!!，代表本設計完成正確的 Draw Bounding Box 功能。

總結而言，本設計的 drawBbox.v 透過 FSM 分階段控制整個影像處理流程，並搭配 UrSRAM、AnsSRAM、3-bank line buffer、Otsu threshold、8-connected CCL 與 bounding box overlay，完成從原始 RGB image 到 final green bounding box image 的完整硬體實作。

最後，系統會將原始 RGB 影像複製到最終輸出記憶體，並根據 bounding box 座標，在每個有效物件周圍畫出 1-pixel 寬的綠色邊界線。若物件碰到影像邊界，則依照 Lab7 規格不繪製 bounding box。當所有 pixel 處理完成後，drawBbox.v 會將 done 訊號拉高，通知 testbench 最終輸出影像已經準備完成，可以開始與 golden data 進行逐 pixel 比對。

- Know the basic design rules
- All operations are activated on the positive edge of the clock.
- Control signals:
 - *Img_CEN*: Active low read enable signal for ImgROM
 - *Ans_CEN*: Active low chip enable signal for AnsSRAM
 - *Ans_WEN*: (Low write/high read) enable signal for ResultSRAM
 - *Ur_CEN*: Active low chip enable signal for UrSRAM
 - *Ur_WEN*: (Low write/high read) enable signal for UrSRAM
 - *done*: Stop the process

本設計遵守 Lab7 所規定的基本設計規則。所有暫存器與狀態轉移皆在 clk 的 positive edge 觸發，因此 FSM controller、address counter、memory control signals 以及內部暫存資料皆以同步方式更新。這樣可以確保整體設計具有明確的時序行為，也有利於後續 synthesis 與 gate-level simulation。

在記憶體控制方面，ImgROM、AnsSRAM 與 UrSRAM 均使用 active-low chip enable 訊號。當 *Img_CEN* 為 0 時，ImgROM 會根據 *Img_A* 所指定的 address 輸出 32-bit RGB pixel；當 *Img_CEN* 為 1 時，ImgROM 不被啟用。由於 ImgROM 只提供讀取功能，因此不需要 *WEN* 訊號。

AnsSRAM 與 UrSRAM 則同時具有讀取與寫入功能。其 *CEN* 訊號同樣為 active low，當 *Ans_CEN* 或 *Ur_CEN* 為 0 時，對應的 SRAM 才會被啟用。*WEN* 訊號用來決定讀寫方向，其中 *WEN* = 0 表示 write operation，*WEN* = 1 表示 read operation。因此，在寫入最終影像到 AnsSRAM 時，需要設定 *Ans_CEN* = 0 且 *Ans_WEN* = 0；在讀取 UrSRAM 中間資料時，則需要設定 *Ur_CEN* = 0 且 *Ur_WEN* = 1。

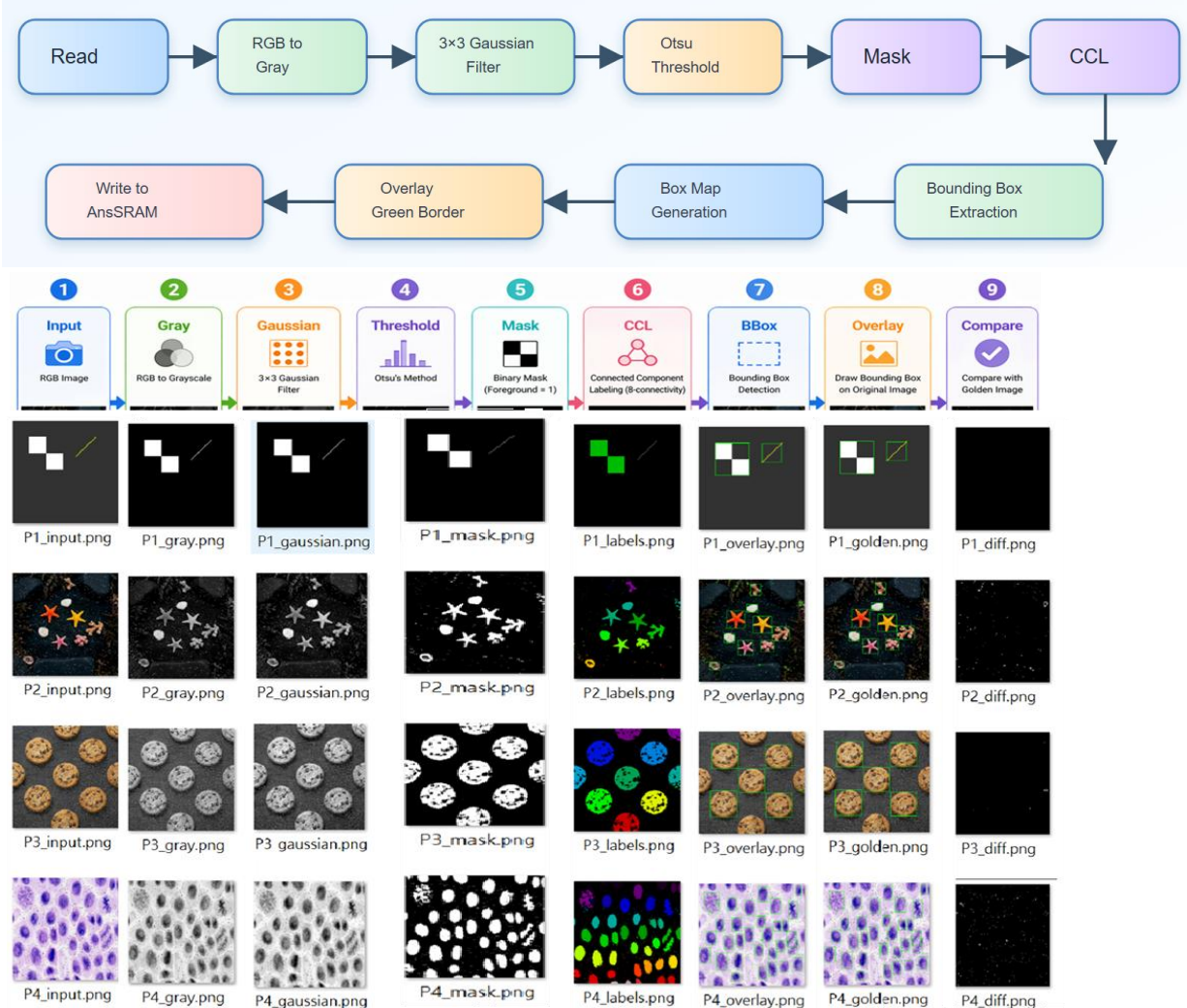
最後，*done* 訊號用來通知 testbench 本設計已完成所有影像處理流程。當 *done* = 1 時，代表 drawBbox.v 已完成 RGB 讀取、灰階轉換、Gaussian filter、Otsu threshold、CCL、bounding box extraction 與 green overlay，testbench 便會開始讀取 AnsSRAM 中的最終結果，並與 golden data 進行逐 pixel comparison。

➤ Describe your design in detail. You can draw internal architecture or block diagram to help elaborate your design, if don't, plain text description is allowed.

A:設計架構詳細說明

本設計為 **Draw Bounding Box** 影像處理硬體架構，主要目標是以 Verilog RTL 實作一個完整的影像前處理、物件偵測與邊界框繪製系統。輸入影像為 **256×256 RGB 彩色影像**，每一個像素由 ImgROM 以 32-bit 資料格式讀入，並依照 row-major order 由左至右、由上至下掃描整張影像。根據 Lab7 規格，本設計需要完成 RGB 轉灰階、Gaussian filter、Otsu thresholding、binary mask 產生、auto-polarity correction、8-connected CCL、bounding box 座標計算，以及最後的綠色邊界框 overlay。Lab7 規格中也明確說明輸入影像大小為 256×256 RGB photo，灰階公式採用 $\text{Gray} \approx 0.299R + 0.587G + 0.114B$ ，Gaussian 邊界處理採用 Reflect 101 padding，threshold 則採用 Otsu's method。

Algorithm Pipeline Implemented in Hardware



Lab7 P1~P2 Draw Bounding Box Engine 之 Python 逐階段影像處理與驗證流程

一、影像讀取與記憶體控制

本設計的第一個階段是從 **ImgROM** 讀取輸入影像。由於影像大小固定為 256×256 ，因此總共有 65,536 個 pixel。系統使用 16-bit address counter 從位址 0 掃描到 65535，依序讀取所有像素。因為影像寬度正好是 256，因此 address 可以很自然地對應到二維座標：

$x = \text{address}[7:0]$

$y = \text{address}[15:8]$

其中 x 代表水平座標，範圍為 0 到 255； y 代表垂直座標，範圍同樣為 0 到 255。這樣的位址轉換方式可以簡化硬體設計，不需要額外除法器或取餘數運算。在讀取階段，控制 FSM 會啟動 active-low 的 **Img_CEN** 訊號，使 **ImgROM** 輸出目前位址對應的 32-bit RGB pixel。讀入後的原始 RGB pixel 會被保留下來，供最後 overlay 階段使用。這樣做的原因是最後輸出不能只輸出 binary mask 或 label 結果，而是要在原始彩色影像上疊加綠色 bounding box。

二、RGB 轉灰階模組

讀入 RGB pixel 後，系統會將彩色影像轉換成 8-bit grayscale image。灰階轉換公式如下：

$$\text{Gray} \approx 0.299R + 0.587G + 0.114B$$

在硬體實作上，為了避免使用浮點數運算，本設計可採用 fixed-point arithmetic 近似計算。例如將係數轉換成整數權重，再透過乘法、加法與位移完成灰階轉換。此階段的目的，是將三通道 RGB 資料轉成單一亮度值，使後續 Gaussian filter 與 Otsu threshold 可以只針對灰階強度進行處理。

灰階模組也需要注意 rounding rule。根據 Lab7 規格，灰階計算結果需要採用 round to nearest, ties to even 的方式處理，以確保輸出結果與 golden data 一致。若 rounding 方法不同，即使肉眼看起來影像差異很小，也可能造成 pixel comparison 失敗。

三、Gaussian Filter 模組

完成灰階轉換後，灰階影像會進入 **3×3 Gaussian filter**。Gaussian filter 的主要功能是降低影像中的雜訊，使後續 thresholding 更穩定。若直接對未平滑的影像做 Otsu thresholding，局部雜訊可能會導致 histogram 分布不穩定，進而造成前景與背景分類錯誤。

Gaussian filter 會根據目前 pixel 周圍的 3×3 鄰域進行加權平均。此設計特別需要注意邊界處理，因為影像邊界 pixel 沒有完整的 3×3 鄰居。Lab7 規定 Gaussian 邊界採用 **Reflect 101 padding**，也就是反射邊界但不重複邊界點。這種方式比 zero-padding 更自然，不會在影像四周產生不合理的黑色邊界，也能讓邊界附近的平滑結果更接近真實影像內容。

公式如下：

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

$$G(x, y) = \frac{1}{16} \sum_{i=-1}^1 \sum_{j=-1}^1 K'(i, j) I(x + i, y + j)$$

四、Histogram 與 Otsu Threshold 模組

Gaussian filter 完成後，系統會根據濾波後的 grayscale image 建立 **256-level histogram**。每一個 gray level 從 0 到 255 都有一個對應的 counter，用來統計該灰階值在整張影像中出現的次數。

接著 Otsu threshold 模組會掃描可能的 threshold value，並計算每個 threshold 對應的 between-class variance。Otsu 的核心概念是找出一個 threshold，使背景與前景兩群之間的差異最大。其公式可表示為：

$$\sigma_B^2 = W_b \times W_f \times (\mu_b - \mu_f)^2$$

其中 W_b 與 W_f 分別代表背景與前景所占比例， μ_b 與 μ_f 則分別代表背景與前景的平均灰階值。當 σ_B^2 最大時，代表該 threshold 能夠最有效地分離背景與前景，因此會被選為最終 threshold。

Lab7 文件中也說明，本實驗的灰階層級為 256，且 binary mask 的判斷規則為：若 gray level 大於 threshold，則該 pixel 為 foreground。

五、Binary Mask 與 Auto-Polarity Correction

取得 Otsu threshold 後，系統會將每一個 Gaussian-filtered gray pixel 與 threshold 比較，產生 binary mask。判斷規則如下：

if gray > T:

mask = 1

else:

mask = 0

其中 mask = 1 代表 foreground pixel，mask = 0 代表 background pixel。

不過，在某些影像中，threshold 後的結果可能會出現前景與背景相反的情況，也就是背景被判定為 foreground，而真正的物件反而變成 background。為了解決這個問題，本設計加入 **Auto-Polarity Correction**。此模組會根據 mask 中 foreground pixel 的分布比例或邊界背景情況，判斷是否需要反轉 mask。

若系統判斷目前的 foreground 區域過大，或背景被錯誤標成主要 foreground，則會進行 polarity flip，使真正的物件成為 foreground。這個步驟非常重要，因為如果不修正極性，後面的 CCL 可能會把整個背景當成一個巨大 connected component，導致 bounding box 錯誤。

六、8-Connected CCL 模組

Binary mask 修正完成後，系統進入 **Connected Component Labeling**，簡稱 **CCL** 階段。CCL 的目的，是將相連的 foreground pixels 分成不同物件，並給每一個 connected component 一個 label。

Lab7 與前一個 Lab 不同，本次要求使用 **8-connectivity**。也就是說，除了上下左右相鄰之外，對角線相鄰的 pixel 也必須被視為同一個物件的一部分。掃描時，由於系統是由左至右、由上至下處理，所以每個 foreground pixel 只需要檢查已經掃描過的四個鄰居：

NW, N, NE, W

如果這四個鄰居都沒有 label，表示目前 pixel 是一個新 component 的起點，因此分配新的 label。若其中有一個或多個鄰居已有 label，則目前 pixel 會繼承其中一個有效 label。若不同鄰居具有不同 label，代表這些 label 其實屬於同一個 connected component，因此需要在 equivalence table 或 union-find 結構中記錄它們的等價關係。

第一階段掃描完成後，系統會進行 label equivalence resolution，將等價的 provisional labels 合併成同一個 root label。這樣可以確保同一個物件內部的所有 pixel 都具有相同的最終 label。Lab7 文件也明確指出，本實驗需使用 8-connectivity，且 8-connected CCL 在 pass 1 需要讀取 NW、N、NE、W 四個方向的鄰居。

七、Bounding Box 座標計算

完成 CCL 後，系統會根據每一個 connected component 的 label 計算 bounding box。對於每一個有效 label，系統會維護四個座標值：

Xmin：該物件最左側 x 座標

Xmax：該物件最右側 x 座標

Ymin：該物件最上方 y 座標

Ymax：該物件最下方 y 座標

當系統掃描到某一個 foreground pixel 時，會根據該 pixel 的 label 更新對應 component 的 bounding box。如果目前 pixel 的 x 座標小於原本的 Xmin，則更新 Xmin；如果 x 座標大於原本的 Xmax，則更新 Xmax。Ymin 與 Ymax 也使用相同方式更新。

透過這種方式，系統可以得到每個物件的最小外接矩形。Lab7 規格中也說明，透過 connected component labeling 可以找出同一 label 的 Xmin、Xmax、Ymin、Ymax。

八、Border Object Rejection

在繪製 bounding box 之前，系統會先檢查該物件是否碰到影像邊界。根據 Lab7 規格，如果物件碰到 border，就不需要畫 bounding box。這是因為邊界物件可

能是不完整物件，或是背景雜訊造成的假物件，因此應該被排除。

硬體中可以透過以下條件判斷：

$X_{min} == 0$

$X_{max} == 255$

$Y_{min} == 0$

$Y_{max} == 255$

只要其中任一條件成立，就代表該 component 接觸到影像邊界，因此該物件會被標記為 invalid，不會進入最後 overlay 繪製階段。

九、Green Bounding Box Overlay

最後一個階段是將 bounding box 疊加回原始 RGB 影像。系統會再次掃描每一個 pixel，並檢查目前座標是否位於任何有效 bounding box 的邊界上。若符合以下任一條件，表示該 pixel 位於 bounding box 邊線上：

$x == X_{min} \text{ and } Y_{min} \leq y \leq Y_{max}$

$x == X_{max} \text{ and } Y_{min} \leq y \leq Y_{max}$

$y == Y_{min} \text{ and } X_{min} \leq x \leq X_{max}$

$y == Y_{max} \text{ and } X_{min} \leq x \leq X_{max}$

若目前 pixel 位於 bounding box 邊界，輸出資料會被替換成綠色常數：

32'h0000_FF00

如果目前 pixel 不在任何 bounding box 邊界上，則輸出原始 RGB pixel。如此一來，最後輸出的 AnsSRAM 影像會保留原圖內容，並只在偵測到的有效物件周圍畫出綠色框線。Lab7 文件也指定 bounding box 應該使用 green overlay，且 green constant 為 32'h0000_FF00。

十、Write to AnsSRAM 與整體輸出控制

完成 Green Bounding Box Overlay 後，系統最後會進入 **Write to AnsSRAM** 階段，將處理後的 256×256 影像結果依照 row-major order 寫入 AnsSRAM。參考附圖「Algorithm Pipeline Implemented in Hardware」，整體硬體流程由 Read 開始，依序經過 RGB to Gray、3×3 Gaussian Filter、Otsu Threshold、Mask、CCL、Bounding Box Extraction、Box Map Generation、Overlay Green Border，最後才進入 Write to AnsSRAM。因此 AnsSRAM 寫入階段不是單純輸出資料，而是整合所有前面模組處理結果的最後一步。

在此階段中，控制器會再次依照 pixel address 從 0 掃描到 65535，並判斷目前 pixel 是否位於 bounding box map 所標記的邊界位置。若目前座標屬於有效 bounding box 的上、下、左、右邊界，系統會將輸出資料設定為綠色常數 32'h0000_FF00；若不是邊界位置，則輸出原始 RGB pixel。寫入 AnsSRAM 時，Ans_CEN 需拉低以啟用 SRAM，Ans_WEN 需設為 0 表示 write operation，Ans_A 則提供目前像素位址，Ans_D 則放入要寫入的 32-bit pixel 資料。Lab7 規格也明確指出 AnsSRAM 的 CEN 為 active low，且 WEN = 0 代表寫入、

WEN = 1 代表讀取；Bounding Box overlay 則需以綠色常數 32'h0000_FF00 表示。

當全部 65,536 個 pixel 都成功寫入 AnsSRAM 後，控制 FSM 會進入完成狀態並將 done 訊號拉高，通知 testbench 可以開始讀取 AnsSRAM 並與 golden image 進行 pixel comparison。此階段的正確性會直接影響最終輸出影像是否能通過 RTL simulation 與 gate-level simulation，因此除了計算 bounding box 正確之外，寫入位址、寫入資料與記憶體控制訊號也必須完全對齊。整體而言，Write to AnsSRAM 是整個 drawBbox pipeline 的收尾模組，負責將前面所有影像處理與物件偵測結果轉換成最終可驗證的輸出影像。

十一、總結

整體而言，本設計從 ImgROM 讀取 256×256 RGB 影像，經過灰階轉換、Gaussian 平滑、Otsu 自動門檻值計算、binary mask 產生、auto-polarity correction、8-connected CCL、bounding box 座標萃取與 border object rejection，最後將有效物件以綠色框線疊加回原始影像並輸出至 AnsSRAM。此架構符合 Lab7 對 pixel correctness、8-connectivity、green bounding box overlay 以及 border object rejection 的要求，也方便後續進行 RTL simulation、gate-level simulation、SuperLint 與 report 說明。

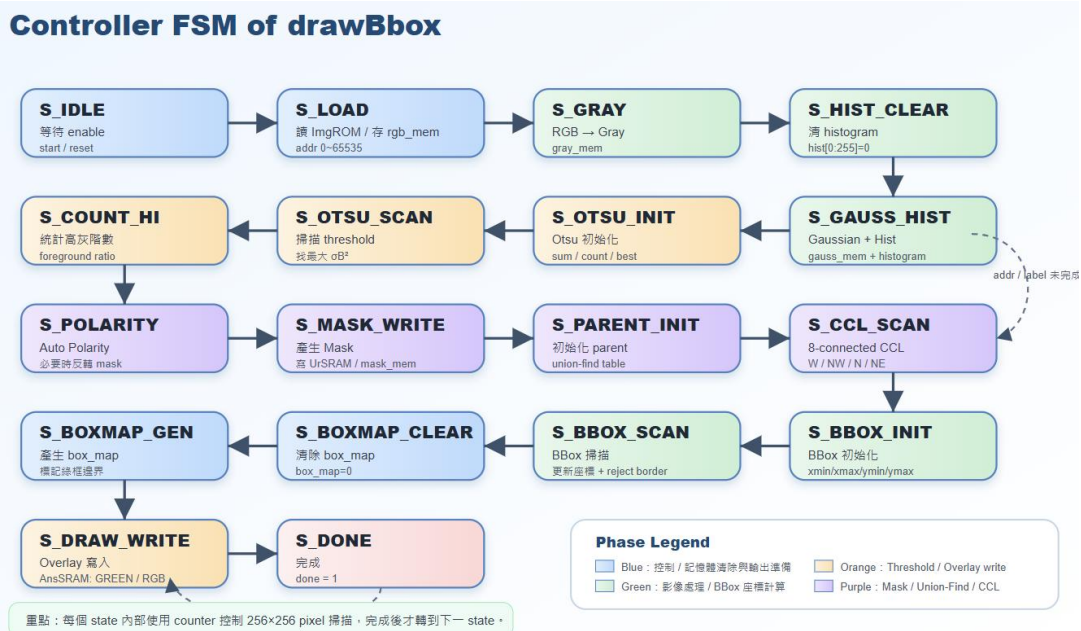
■ Controller

◆ Draw your state diagram in controller and explain it.

A: 本設計使用有限狀態機作為 drawBbox.v 的主要控制器。Controller 負責依序控制 ImgROM、UrSRAM 與 AnsSRAM 的讀寫時序，並將完整影像處理流程拆成多個 state 執行。這樣的設計可以避免將所有影像處理演算法集中在單一大型 combinational block 中，也能讓 Gaussian filter、Otsu threshold、CCL 與 bounding box overlay 等功能分階段完成，使設計更容易模擬、除錯與合成。

State diagram 說明

本設計的 FSM 主要流程如下：



其中 S_LB_ROW 會在 Gaussian 階段跨列更新 line buffer 後回到 S_GAUSS_HIST，而 S_LB_ROW2 會在 CCL 階段跨列更新 line buffer 後回到 S_CCL_SCAN。這兩個狀態主要用來支援逐列掃描時的 3-row line buffer 更新。

各 state 功能說明

State	功能說明
S_IDLE	等待 enable 訊號。當 enable = 1 時，FSM 開始執行影像處理流程。
S_LOAD	從 ImgROM 讀取 256×256 RGB pixel。原始 RGB 寫入 AnsSRAM 作為後續 overlay 底圖，同時透過 gray_round_rgb() 轉成 gray 後寫入 UrSRAM。
S_HIST_CLEAR	清除 256-bin histogram，並初始化 Gaussian 與 Otsu 計算所需的計數器與累加器。
S_LB_INIT	從 UrSRAM 載入初始 3-row line buffer，提供 Gaussian filter 使用。

S_GAUSS_HIST	使用 3×3 Gaussian filter 計算平滑後的 gray value，同時建立 histogram 並累加 sum_total。
S_LB_ROW	每處理完一列影像後，更新下一列 line buffer，完成後回到 S_GAUSS_HIST 繼續處理下一列。
S_OTSU_INIT	初始化 Otsu threshold 計算所需的背景累加值、最佳 threshold 與比較用暫存器。
S_OTSU_SCAN	逐一掃描 gray level，更新背景 pixel 數與背景灰階總和。
S_OTSU_LHS	計算 Otsu 比較式中的左側中間值。
S_OTSU_DENOM	計算分母項與差值，避免在單一 state 中建立過大的運算邏輯。
S_OTSU_SQUARE	計算差值平方，用於比較不同 threshold 的類間變異數。
S_OTSU_CMP1	準備與目前最佳 threshold 的比較運算。
S_OTSU_CMP2	比較目前 threshold 是否較佳，若較佳則更新 otsu_t。若所有 threshold 掃描完成，進入 S_COUNT_HI。
S_COUNT_HI	統計高於 Otsu threshold 的 pixel 數量，用於判斷是否需要 auto-polarity correction。
S_POLARITY	根據 count_hi 判斷是否設定 invert_sel。若高亮 pixel 超過半張影像，代表可能需要反相 foreground/background。
S_PARENT_INIT	初始化 CCL 所需的 parent[]、bbox_valid[]、bbox_border[]、bbox_xmin[]、bbox_xmax[]、bbox_ymin[] 與 bbox_ymax[]。
S_LB_INIT2	在 CCL 前重新載入 line buffer，使 CCL 掃描時能即時計算 foreground pixel。
S_CCL_SCAN	執行 8-connected CCL。對每個 foreground pixel 檢查 NW、N、NE、W 四個已掃描鄰居，並透過 union_pair() 合併等價 label，同時更新 bounding box 座標。
S_LB_ROW2	CCL 跨列時更新 line buffer，完成後回到 S_CCL_SCAN 繼續下一列掃描。
S_BBOX_INIT	將等價 label 的 bounding box 合併到 root label，確保每個 connected component 都有正確的 xmin、xmax、ymin、ymax。
S_BOXMAP_GEN	依序檢查每個有效 bounding box，排除無效、邊界物件或太小的物件，並將有效 bbox 的四條邊寫成綠色 32'h0000_FF00。
S_DONE	將 done 訊號拉高，通知 testbench 最終影像已完成，

	可開始與 golden data 進行逐 pixel comparison。
--	--

總結

本設計的 Controller 以 FSM 方式控制完整影像處理流程。首先在 S_LOAD 中完成原始 RGB 讀取、灰階轉換與記憶體儲存，接著透過 S_GAUSS_HIST 建立 Gaussian-filtered image 的 histogram。Otsu threshold 計算被拆成多個 state，以降低單一 clock cycle 的運算負擔並提升 synthesis 穩定性。完成 threshold 與 polarity 判斷後，FSM 進入 CCL 階段，使用 8-connected CCL 標記 connected components 並即時更新 bounding box。最後在 S_BOXMAP_GEN 中將有效 bounding box 畫回 AnsSRAM，完成後進入 S_DONE 並將 done 拉高。

1.Complete the bicubic module, in the system.

本設計已完成 Lab7 系統中的主要設計模組 drawBbox.v。雖然模板中使用“bicubic module”的敘述，但在本次 Draw Bounding Box Engine 作業中，實際需要完成的是 drawBbox module。本設計在不修改原始 I/O port 的前提下，完成與 testbench、ImgROM、UrSRAM 及 AnsSRAM 的連接，並實作完整的影像處理流程。

系統啟動後，drawBbox.v 會先透過 Img_CEN 與 Img_A 從 ImgROM 讀取 256×256 的 32-bit RGB pixel。讀取資料後，系統會將原始影像寫入 AnsSRAM 作為後續 overlay 的基礎，同時將 RGB pixel 轉換成灰階資料並寫入 UrSRAM。灰階轉換使用近似公式：

$$\text{Gray} \approx 0.299R + 0.587G + 0.114B$$

在硬體實作中，使用整數權重 77、150、29 進行 fixed-point 計算，並加入 round-to-nearest, ties-to-even 的 rounding 判斷，以接近 Lab7 規格要求。

接著，系統使用 3-row line buffer 進行 3×3 Gaussian filter。為了避免在硬體中建立過大的 full-image register array，本設計將灰階影像儲存在 UrSRAM 中，並使用三列 line buffer 配合 bank selector 進行 Gaussian 計算。Gaussian kernel 為：

```
1 2 1
2 4 2
1 2 1
```

因為權重總和為 16，因此除法可用 right shift 取代，以降低硬體成本。邊界處理則使用 Reflect 101 padding，透過 refl101() function 修正邊界座標。

完成 Gaussian filter 後，系統會建立 256-bin histogram，並使用 Otsu method 計算最佳 threshold。為了降低合成時的硬體複雜度，Otsu 計算沒有集中在單一大型 combinational block 中，而是拆成多個 FSM state，例如 S_OTSU_SCAN、S_OTSU_LHS、S_OTSU_DENOM、S_OTSU_SQUARE、S_OTSU_CMP1 與 S_OTSU_CMP2。這樣可以共用乘法器資源，降低 Design Compiler 合成時的負擔。

在 binary mask 與 auto-polarity correction 階段，系統會根據 Otsu threshold 判斷 foreground pixel。若 threshold 後的高亮 pixel 數量過多，代表可能將背景誤判為前景，因此設置 invert_sel 進行 polarity correction。

CCL 階段使用 8-connected connected component labeling。系統以 raster scan 方式掃描整張影像，對每個 foreground pixel 檢查已掃描過的四個鄰居：

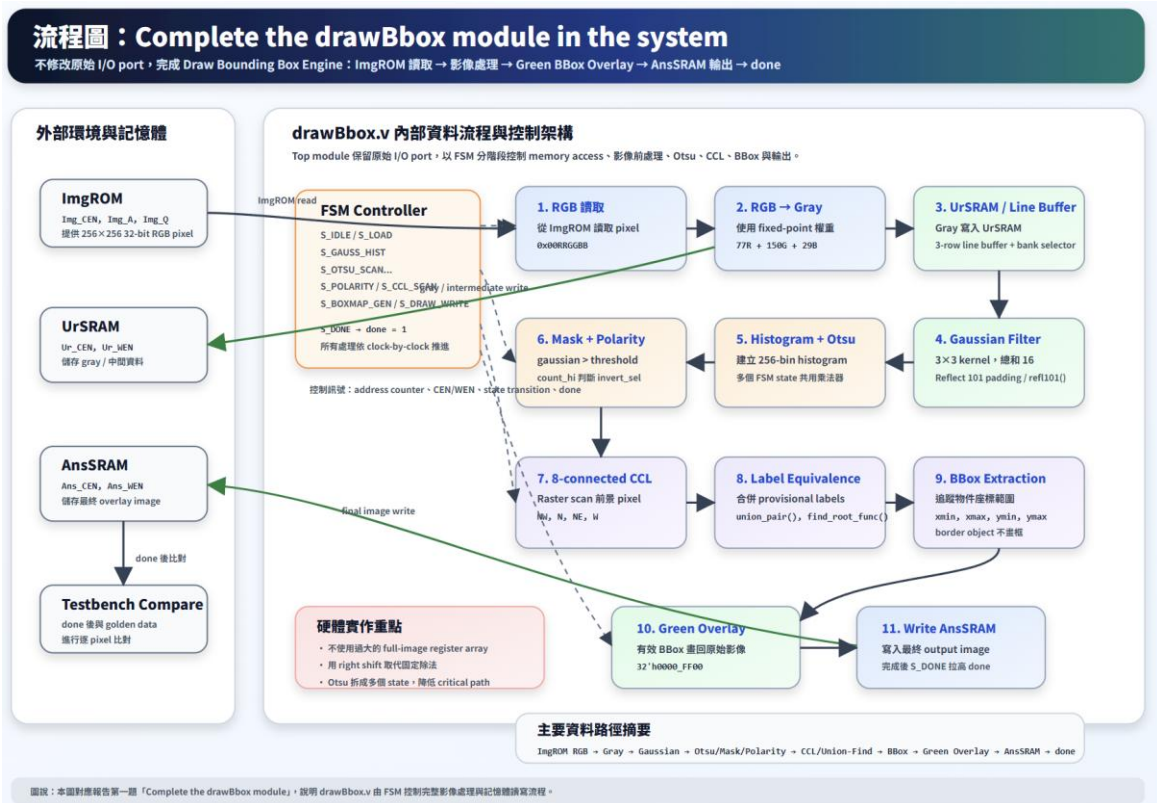
NW, N, NE, W

若不同 label 屬於同一 connected component，則使用 union_pair() 建立等價關係，並透過 find_root_func() 找出 root label。接著，系統更新每個 label 對應

的 bounding box 座標，包括 xmin、xmax、ymin 與 ymax。若物件接觸影像邊界，則設定為 border object，不繪製 bounding box。

最後，系統在 S_BOXMAP_GEN 與 S_DRAW_WRITE 等階段將有效 bounding box 畫回原始影像。Bounding box 的顏色使用 Lab7 指定的綠色常數：32'h0000_FF00

當所有 pixel 完成處理並寫入 AnsSRAM 後，FSM 進入 S_DONE，並將 done 訊號拉高，通知 testbench 可以開始進行 golden data comparison。

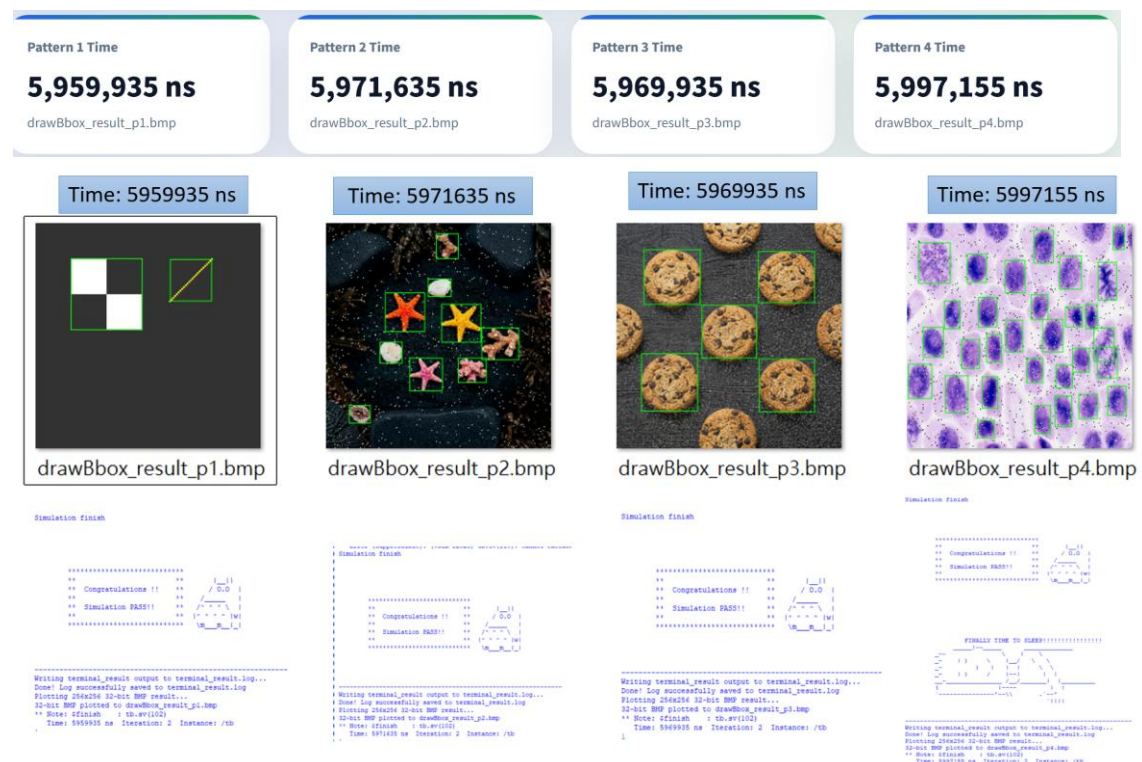


2. Compile the verilog code to verify the operations of this module works properly.

本題主要說明本設計已完成 Verilog code 編譯與 RTL simulation 驗證，確認 drawBbox.v 在 ModelSim 中可以正常運作。驗證時使用課程提供的 tb.sv 作為 testbench，並分別執行 Pattern 1 到 Pattern 4。testbench 會讀取對應的 input image，等待 drawBbox.v 完成影像處理並將 done 訊號拉高後，再將 AnsSRAM 中的輸出結果與 golden data 進行逐 pixel 比對。

設計的 RTL simulation 以 ModelSim 執行。模擬過程中，drawBbox.v 會依序完成 ImgROM 讀取、RGB to Gray、Gaussian filter、Otsu threshold、binary mask、auto-polarity correction、8-connected CCL、bounding box extraction、green overlay，以及 AnsSRAM 輸出。若輸出結果與 golden data 完全一致，terminal 會顯示 Congratulations !! Simulation PASS!!，並產生對應的 drawBbox_result_p*.bmp 影像檔。

從模擬結果可以看到，Pattern 1 到 Pattern 4 皆成功完成 RTL simulation，且 terminal 皆顯示 Simulation PASS!!。四組 pattern 也都成功產生對應的 BMP 輸出影像，分別為 drawBbox_result_p1.bmp、drawBbox_result_p2.bmp、drawBbox_result_p3.bmp 與 drawBbox_result_p4.bmp。輸出影像中可以看到綠色 bounding box 已正確標示在偵測到的物件周圍，表示本模組的功能可以正常運作。



Lab7 Draw Bounding Box Engine 在 ModelSim 中之 RTL Simulation PASS 結果

Pattern	輸出檔案	RTL Simulation Time	Terminal Result	驗證說明
Pattern 1	drawBbox_result_p1.bmp	5,959,935 ns	✓ PASS	成功處理簡單幾何物件，並畫出 green bounding box。
Pattern 2	drawBbox_result_p2.bmp	5,971,635 ns	✓ PASS	成功偵測多個不同形狀物件，並完成多個 bounding boxes。
Pattern 3	drawBbox_result_p3.bmp	5,969,935 ns	✓ PASS	成功辨識餅乾物件，並排除或處理邊界相關物件。
Pattern 4	drawBbox_result_p4.bmp	5,997,155 ns	✓ PASS	成功處理較複雜的細胞影像，並畫出多個 green bounding boxes。

本次 RTL simulation 結果顯示，四組測試圖皆可成功通過 ModelSim 模擬，且輸出影像與 golden data 比對結果正確。因此，可以確認 drawBbox.v 的基本功能已完成，並且各階段影像處理流程能在 RTL level 正常運作。後續即可進一步進行 synthesis、SuperLint 檢查與 post-synthesis simulation，以確認合成後的 gate-level design 仍能維持相同功能正確性。

3. Synthesize your *drawBbox.v* with following constraint:

- Clock period: no more than 10.0 ns.
- Don't touch network: clk.
- Wire load model: N16ADFP_StdCellss0p72vm40c.
- Synthesized verilog file: drawBbox_syn.v.
- Timing constraint file: drawBbox_syn.sdf.

drawBbox.v 進行邏輯合成，並依照課程指定的 synthesis constraints 產生後續 gate-level simulation 所需的檔案。指定條件如下：

項目	題目要求	本設計設定 / 結果
Clock period	no more than 10.0 ns	設定為 10.0 ns
Don't touch network	clk	在 DC constraint 中保護 clock network
Wire load model	N16ADFP_StdCellss0p72vm40c	依題目指定 library / wire load model 設定
Synthesized Verilog file	drawBbox_syn.v	已產生
Timing constraint file	drawBbox_syn.sdf	已產生

二、合成設定說明

本設計使用 Design Compiler 對 drawBbox.v 進行 synthesis。合成時，clock period 設定為 10.0 ns，符合題目中「clock period no more than 10.0 ns」的限制。此設定會使 synthesis tool 以 10.0 ns 作為 timing constraint，嘗試最佳化 combinational path，使 critical path delay 能滿足指定時脈週期。

在 clock network 的設定上，本設計依照題目要求將 clk 設定為 don't touch network。此設定的目的，是避免 synthesis tool 對 clock signal 進行不必要的邏輯最佳化或結構修改，使 clock path 能維持穩定，並確保後續 gate-level simulation 與 timing annotation 時，clock behavior 不會因合成最佳化而產生非預期問題。

Wire load model 則依題目指定使用 N16ADFP_StdCellss0p72vm40c。此 library / wire load model 用於 synthesis 時估計 standard cell 與 interconnect 的負載與延遲，使 timing analysis 能符合課程指定的製程與標準元件環境。

三、產生檔案說明

完成 synthesis 後，本設計成功產生下列檔案：

產生檔案	說明	用途
drawBbox_syn.v	合成後的 gate-level Verilog netlist	用於 post-synthesis / gate-level simulation

drawBbox_syn.sdf	合成後的 SDF timing delay file	用於 gate-level simulation 的 timing annotation
------------------	----------------------------	--

其中，drawBbox_syn.v 是由 RTL Verilog 轉換後的 gate-level netlist，內部會由 standard cells 組成，不再只是原本的 behavioral RTL 描述。drawBbox_syn.sdf 則記錄合成後 cell delay 與 timing delay，在 post-synthesis simulation 時會被 testbench 或 simulation flow 讀入，用來模擬更接近實際硬體的 timing behavior。

四、合成結果與檔案截圖說明

根據目前 synthesis output 資料夾中的結果，可以看到已成功產生 drawBbox_syn.v 與 drawBbox_syn.sdf。其中 drawBbox_syn.v 為 V 檔，大小約 124,675 KB；drawBbox_syn.sdf 為 SDF timing file，大小約 2,023,716 KB。這表示 drawBbox.v 已完成 Design Compiler synthesis，並輸出後續 gate-level simulation 所需的 netlist 與 timing annotation file。

名稱	修改日期	類型	大小
drawBbox_syn.sdf	2026/5/5 下午 05:55	SQL Server Compac...	2,023,716 KB
drawBbox.fsdb	2026/5/5 下午 05:55	FSDB 檔案	1,089,452 KB
drawBbox_syn.v	2026/5/5 下午 05:54	V 檔案	124,675 KB
drawBbox.fsdb.xlist	2026/5/5 下午 05:54	XLIST 檔案	13,016 KB
drawBbox.fsdb.chain	2026/5/5 下午 05:54	CHAIN 檔案	5,088 KB
simv	2026/5/5 下午 05:54	檔案	2,823 KB
drawBbox.fsdb.spchain	2026/5/5 下午 05:54	SPCHAIN 檔案	2,561 KB
drawBbox.fsdb.field	2026/5/5 下午 05:54	FIELD 檔案	1,729 KB
drawBbox.fsdb.slist	2026/5/5 下午 05:54	SLIST 檔案	1,690 KB
command.log	2026/5/5 下午 05:55	文字文件	316 KB
drawBbox_result_p4.bmp	2026/5/5 下午 05:54	BMP 檔案	257 KB
default.svf	2026/5/5 下午 05:55	SVF 檔案	256 KB
drawBbox.fsdb.netwk	2026/5/5 下午 05:54	NETWK 檔案	84 KB
drawBbox.fsdb.pidat	2026/5/5 下午 05:54	PIDAT 檔案	67 KB
drawBbox.v	2026/5/5 下午 05:54	V 檔案	45 KB

圖 X drawBbox.v 合成後產生之輸出檔案。圖中可看到已產生 drawBbox_syn.v 與 drawBbox_syn.sdf，分別作為後合成 gate-level simulation 的 netlist 與 timing annotation file。

五、合成流程說明

本設計合成流程如下：

步驟	說明
1	讀入 RTL 原始碼 drawBbox.v
2	讀入 synthesis constraint DC.sdc
3	設定 clock period 為 10.0 ns
4	設定 clk 為 don't touch network
5	設定 library / wire load model 為 N16ADFP_StdCellss0p72vm40c
6	執行 Design Compiler synthesis
7	產生 drawBbox_syn.v

8	產生 drawBbox_syn.sdf
9	產生 area report 與 timing report
10	使用 drawBbox_syn.v 與 drawBbox_syn.sdf 進行後合成模擬

本設計使用 Design Compiler 對 drawBbox.v 進行邏輯合成。合成時，clock period 設定為 10.0 ns，符合題目要求中「clock period no more than 10.0 ns」的限制。此設定使 synthesis tool 以 10.0 ns 作為主要 timing constraint，並嘗試最佳化設計中的 critical path，使合成後電路能符合指定時脈週期。

在 clock network 設定方面，本設計依照題目要求將 clk 設定為 don't touch network，避免 synthesis tool 對 clock signal 進行任意修改或最佳化。此做法可確保 clock path 維持穩定，並降低後續 gate-level simulation 時因 clock network 被改變而造成非預期錯誤的可能性。

Wire load model 依照題目指定使用 N16ADFP_StdCellss0p72vm40c。此設定用於 synthesis 階段的 wire delay 與 loading estimation，使 timing analysis 能符合課程指定之 standard cell library 環境。雖然在實際執行時需要確認 .synopsys_dc.setup 是否正確讀入對應的 .db library，但本次 synthesis output 已成功產生 drawBbox_syn.v 與 drawBbox_syn.sdf，表示合成流程已完成。

合成完成後，系統產生 drawBbox_syn.v 與 drawBbox_syn.sdf 兩個主要檔案。drawBbox_syn.v 是合成後的 gate-level Verilog netlist，會在後續 post-synthesis simulation 中作為設計檔使用；drawBbox_syn.sdf 則為 Standard Delay Format 檔案，記錄合成後電路的 timing delay，用於 gate-level simulation 的 timing annotation。根據目前合成資料夾截圖，可確認兩個指定檔案皆已成功產生。

因此，本題要求的 synthesis constraints 與 output files 均已完成。本設計後續可使用 make syn1、make syn2、make syn3 與 make syn4 進行 Pattern 1 到 Pattern 4 的 post-synthesis simulation，以確認合成後電路仍能與 RTL simulation 一樣通過 golden data comparison。

4. Please attach the resulting **final drawBbox_result_p4.bmp** image generated from the simulation. You may also attach a non-final drawBbox_result.bmp together as well to highlight your progression to a perfect output (not compulsory).

一、P1~P4 Simulation 結果

本題要求附上由 simulation 產生的最終輸出影像 **drawBbox_result_p4.bmp**。此圖應呈現原始 RGB 影像內容，並在有效物件周圍畫出綠色 bounding box

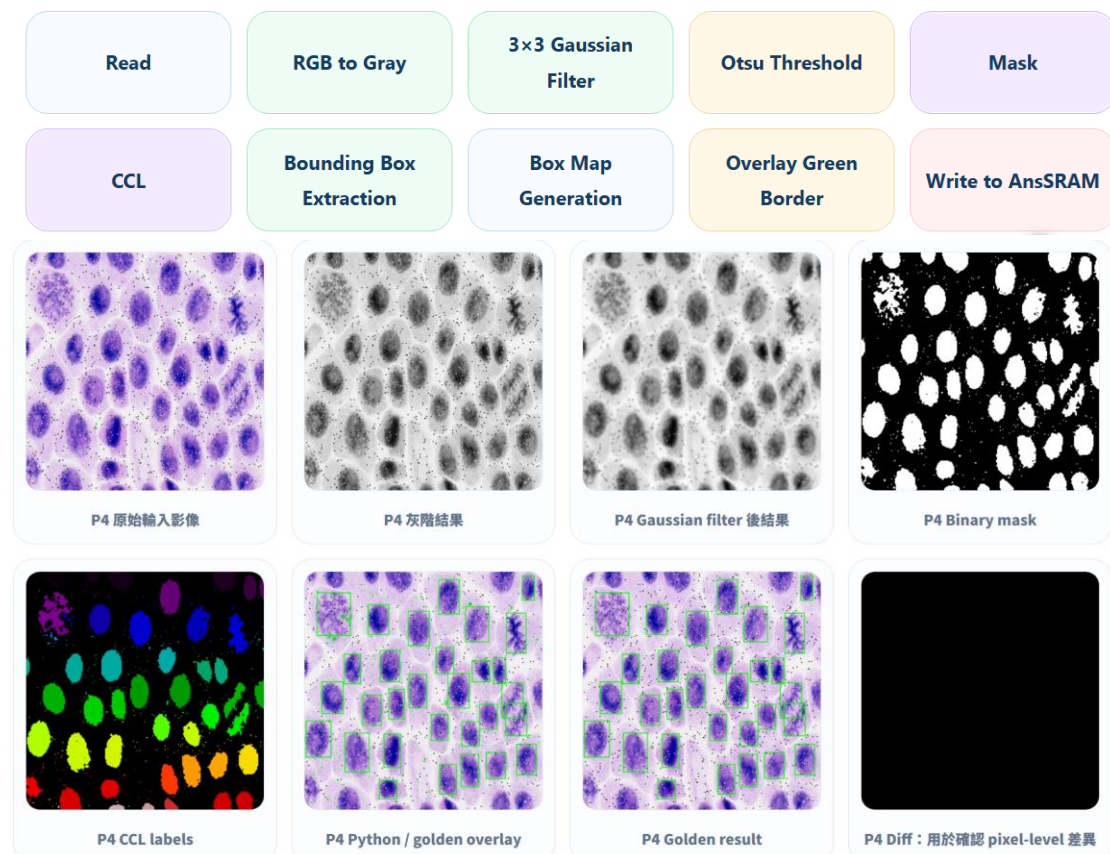


Pattern	Output file	說明
P1	drawBbox_result_p1.bmp	可作為早期 pattern 驗證結果
P2	drawBbox_result_p2.bmp	確認不同 threshold 與物件形狀仍正確
P3	drawBbox_result_p3.bmp	確認多 component 與 bbox 邊界穩定
P4	drawBbox_result_p4.bmp	本題正式 final image

P4 為本設計完成 simulation 後產生的 **drawBbox_result_p4.bmp**。由下圖**模擬 drawBbox_result_p4.bmp 整個流程中間圖展示**中可以觀察到，系統已成功保留原始 RGB 影像內容，並在有效物件周圍繪製綠色 bounding box。此結果顯示本設計的 Otsu threshold、8-connected CCL、bounding box extraction、box map generation 與 overlay 寫回流程皆能正確運作。

二、硬體演算法流程說明

本設計的硬體演算法流程主要包含 Read、RGB to Gray、3×3 Gaussian Filter、Otsu Threshold、Mask、CCL、Bounding Box Extraction、Box Map Generation、Overlay Green Border、Write to AnsSRAM 十個階段。整體目標是從 ImgROM 讀取 256×256 RGB 影像，經過影像前處理與物件偵測後，最後在有效物件周圍畫出綠色 bounding box，並將結果寫入 AnsSRAM。



模擬 drawBbox_result_p4.bmp 整個流程中間圖展示

(一) Read：影像讀取與座標轉換

輸入影像大小為 256×256 ，因此共有 65,536 個 pixel。硬體使用 16-bit address counter 依照 row-major order 由位址 0 掃描到 65535。由於影像寬度為 256，因此位址可以直接拆成 x、y 座標：

$$x = \text{address}[7:0]$$

$$y = \text{address}[15:8]$$

其中 x 為水平座標，範圍為 0 到 255；y 為垂直座標，範圍也為 0 到 255。此方式可避免使用除法器與取餘數電路，降低硬體複雜度。

(二) RGB to Gray：灰階轉換

讀取 RGB pixel 後，系統會將三通道 RGB 資料轉換成單一灰階值。灰階公式如下：

$$\text{Gray} \approx 0.299R + 0.587G + 0.114B$$

在硬體實作上，為避免浮點數運算，可使用 fixed-point arithmetic 近似。例如將係數放大為整數後再右移：

$$\text{Gray} \approx (77R + 150G + 29B) / 256$$

$$\text{Gray} \approx (77R + 150G + 29B) \gg 8$$

其中 $77/256 \approx 0.3008$ ， $150/256 \approx 0.5859$ ， $29/256 \approx 0.1133$ ，與原公式相近。灰階轉換的目的，是將 RGB 三通道影像轉成單一亮度影像，使後續 Gaussian filter、histogram 統計與 Otsu threshold 可以只針對亮度值進行處理。

(三) 3×3 Gaussian Filter：影像平滑化

Gaussian filter 用來降低雜訊，使後續 thresholding 更穩定。本設計使用 3×3 Gaussian kernel，常見權重如下：

$$G = 1/16 \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

對於座標 (x, y) 的 pixel，其濾波後灰階值可表示為：

$$\begin{aligned} \text{Gray_g}(x,y) = & [1 \times \text{Gray}(x-1,y-1) + 2 \times \text{Gray}(x,y-1) + 1 \times \text{Gray}(x+1,y-1) \\ & + 2 \times \text{Gray}(x-1,y) + 4 \times \text{Gray}(x,y) + 2 \times \text{Gray}(x+1,y) \\ & + 1 \times \text{Gray}(x-1,y+1) + 2 \times \text{Gray}(x,y+1) + 1 \times \text{Gray}(x+1,y+1)] / 16 \end{aligned}$$

因為除以 16 可以用右移 4 bits 取代，所以硬體可寫成：

$$\text{Gray_g}(x,y) = \text{weighted_sum} \gg 4$$

邊界 pixel 需要特別處理。本設計採用 Reflect 101 padding，也就是反射邊界但不重複邊界點。例如 $x = -1$ 時，會映射到 $x = 1$ ； $x = 256$ 時，會映射到 $x = 254$ 。這樣可以避免 zero-padding 造成邊界變暗。

(四) Histogram：灰階統計

Gaussian filter 完成後，系統會建立 256-level histogram。每一個 gray level i 都有一個 counter，記錄該灰階值出現次數：

$$h(i) = \text{number of pixels whose gray level} = i, 0 \leq i \leq 255$$

由於影像共有 65,536 個 pixel，因此所有 histogram counter 的總和為：

$$\sum h(i) = 65536, i = 0 \text{ to } 255$$

Histogram 是 Otsu threshold 的基礎，若 histogram 統計錯誤，後續 threshold、mask、CCL 與 bounding box 都會受到影響。

(五) Otsu Threshold：自動門檻值計算

Otsu method 的目標是找出一個 threshold T ，使前景與背景的類間變異數最大。

對於某個 threshold T ，可將灰階分成 background 與 foreground：

$$\text{Background: } 0 \leq i \leq T$$

$$\text{Foreground: } T+1 \leq i \leq 255$$

背景與前景的 pixel 數量分別為：

$$N_b(T) = \sum h(i), i = 0 \text{ to } T$$

$$N_f(T) = \sum h(i), i = T+1 \text{ to } 255$$

背景與前景的權重為：

$$W_b(T) = N_b(T) / N$$

$$W_f(T) = N_f(T) / N$$

$$N = 65536$$

背景與前景的平均灰階值為：

$$\mu_b(T) = [\sum i \times h(i), i = 0 \text{ to } T] / N_b(T)$$

$$\mu_f(T) = [\sum i \times h(i), i = T+1 \text{ to } 255] / N_f(T)$$

Otsu 的類間變異數公式如下：

$$\sigma_B^2(T) = W_b(T) \times W_f(T) \times [\mu_b(T) - \mu_f(T)]^2$$

系統會掃描所有可能的 T ，並選擇使 σ_B^2 最大的 threshold：

$$T_{otsu} = \operatorname{argmax} \sigma_B^2(T)$$

在硬體中，為避免浮點數與除法造成面積過大，也可以使用等價的整數比較形式。令：

$$S_b(T) = \sum i \times h(i), i = 0 \text{ to } T$$

$$S_f(T) = \sum i \times h(i), i = T+1 \text{ to } 255$$

則可用累加值與 pixel 數進行比較，減少直接計算小數權重的需求。

(六) Mask：二值化影像產生

取得 Otsu threshold 後，每個 Gaussian-filtered pixel 會與 threshold 比較，產生 binary mask：

if Gray_g(x,y) > T_otsu:

$$\text{Mask}(x,y) = 1$$

else:

$$\text{Mask}(x,y) = 0$$

其中 Mask = 1 表示 foreground pixel，Mask = 0 表示 background pixel。Binary mask 是後續 CCL 的輸入。

(七) Auto-Polarity Correction：前景極性修正

某些影像中，亮區不一定是真正物件，因此 threshold 後可能產生前景與背景反轉。為避免 CCL 將背景誤判為巨大 component，系統會統計 high pixel 數量：

$$N_{high} = \sum \text{Mask}(x,y)$$

若 high pixel 數量過大，表示目前 foreground 可能是背景，則進行 polarity flip：

$$\text{Mask_corrected}(x,y) = 1 - \text{Mask}(x,y)$$

若 high pixel 數量合理，則維持原 mask：

$$\text{Mask_corrected}(x,y) = \text{Mask}(x,y)$$

(八) CCL：Connected Component Labeling

CCL 會將相連的 foreground pixels 分成不同 connected components。本設計使用 8-connectivity，因此除了上下左右，對角線相鄰也視為同一物件。掃描順序為由左至右、由上至下，因此每個 pixel 只需要檢查已經掃描過的四個鄰居：

$$NW = \text{Label}(x-1, y-1)$$

$$N = \text{Label}(x, y-1)$$

$$NE = \text{Label}(x+1, y-1)$$

$$W = \text{Label}(x-1, y)$$

標記規則如下：

若 $\text{Mask_corrected}(x,y) = 0$:

$\text{Label}(x,y) = 0$

若 $\text{Mask_corrected}(x,y) = 1$ 且 NW,N,NE,W 全為 0 :

$\text{Label}(x,y) = \text{new_label}$

若 $\text{Mask_corrected}(x,y) = 1$ 且存在鄰居 label :

$\text{Label}(x,y) = \min(\text{nonzero neighbor labels})$

不同 label 之間記錄 equivalence

若兩個 label 被判定為同一 connected component，則透過 union-find 或 parent table 合併：

$\text{parent}[\text{root_a}] = \text{root_b}$

合併完成後，所有等價 label 會被解析成相同 root label。

(九) Bounding Box Extraction：邊界框座標計算

對每個 connected component，系統會計算其最小外接矩形。對於 label k，四個座標定義如下：

$X_{\min}(k) = \min\{x \mid \text{Label}(x,y) = k\}$

$X_{\max}(k) = \max\{x \mid \text{Label}(x,y) = k\}$

$Y_{\min}(k) = \min\{y \mid \text{Label}(x,y) = k\}$

$Y_{\max}(k) = \max\{y \mid \text{Label}(x,y) = k\}$

硬體掃描時，每遇到 label k 的 pixel，就更新：

$X_{\min}(k) = \min(X_{\min}(k), x)$

$X_{\max}(k) = \max(X_{\max}(k), x)$

$Y_{\min}(k) = \min(Y_{\min}(k), y)$

$Y_{\max}(k) = \max(Y_{\max}(k), y)$

若 component 接觸影像邊界，則不繪製 bounding box：

if $X_{\min}(k) == 0$ or $X_{\max}(k) == 255$ or $Y_{\min}(k) == 0$ or $Y_{\max}(k) == 255$:

$\text{valid}(k) = 0$

else:

$\text{valid}(k) = 1$

(十) Box Map Generation 與 Green Overlay

有效 bounding box 會被轉換成 box map。若目前 pixel 位於任一有效 bounding box 的四條邊上，則 box map 設為 1：

$\text{BoxMap}(x,y) = 1$ if:

$x == X_{\min}(k)$ and $Y_{\min}(k) \leq y \leq Y_{\max}(k)$

or $x == X_{\max}(k)$ and $Y_{\min}(k) \leq y \leq Y_{\max}(k)$

or $y == Y_{\min}(k)$ and $X_{\min}(k) \leq x \leq X_{\max}(k)$

or $y == Y_{\max}(k)$ and $X_{\min}(k) \leq x \leq X_{\max}(k)$

最後 overlay 階段會根據 BoxMap 決定輸出 pixel：

if $\text{BoxMap}(x,y) == 1$:

```
Output(x,y) = 32'h0000_FF00
else:
    Output(x,y) = Original_RGB(x,y)
```

其中 32'h0000_FF00 為綠色 bounding box pixel。最後系統將 Output(x,y) 依照 row-major order 寫入 AnsSRAM。

(十一) Write to AnsSRAM：結果寫回

寫回階段會將所有 65,536 個 output pixels 寫入 AnsSRAM。記憶體控制訊號如下：

```
Ans_CEN = 0 // 啟用 AnsSRAM
```

```
Ans_WEN = 0 // write mode
```

```
Ans_A    = address
```

```
Ans_D    = Output(x,y)
```

當 address 從 0 寫到 65535 後，代表整張影像輸出完成，FSM 會拉高 done 訊號，通知 testbench 可以讀取 AnsSRAM 並產生最終的 **drawBbox_result_p4.bmp**。

5. Please attach your waveforms and specify your operations on the waveforms.

本設計使用 ModelSim / nWave 觀察 drawBbox.v 的主要控制訊號與記憶體介面訊號，以確認整體影像處理流程是否依照 FSM state 正確執行。Waveform 中主要觀察的訊號包含 clk、rst、enable、done、FSM state、ImgROM 介面訊號、UrSRAM 介面訊號與 AnsSRAM 介面訊號。在 waveform 中，clk 作為整個系統的同步時脈，所有暫存器與 FSM state transition 都在 clock positive edge 更新。當 rst 拉高時，FSM 會回到初始狀態，內部 counter、address、done 與 memory control signals 會被初始化。當 enable 拉高後，FSM 開始從 S_IDLE 進入主要影像處理流程。

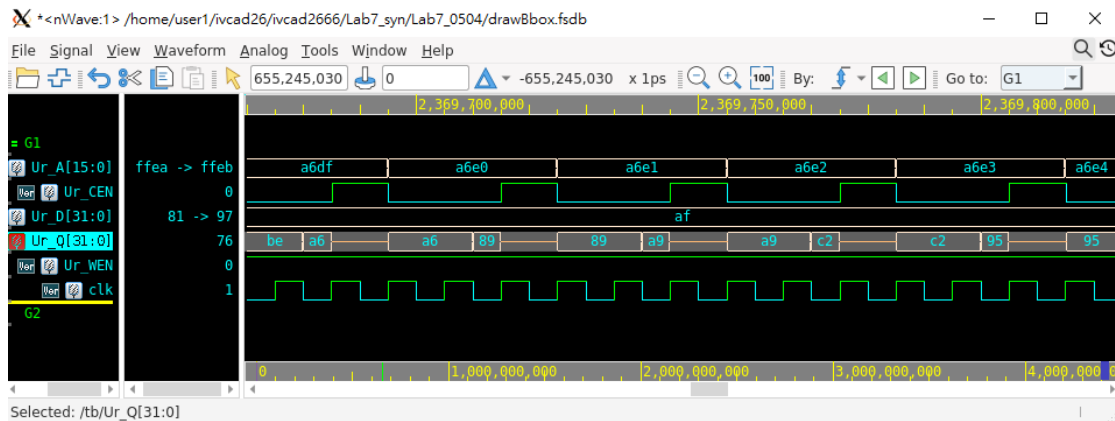


圖 A 顯示 UrSRAM 在 drawBbox.v 中的讀寫控制波形。

圖 A 為 drawBbox.v 在 ModelSim / nWave 中觀察到的 UrSRAM memory access waveform。圖中主要訊號包含 clk、Ur_CEN、Ur_WEN、Ur_A、Ur_D 與 Ur_Q。由於本設計所有操作皆在 clock positive edge 觸發，因此 FSM state transition、address 更新與資料讀寫皆會隨著 clk 同步進行。

在 UrSRAM 控制中，Ur_CEN 為 active low chip enable signal，當 Ur_CEN = 0 時代表 UrSRAM 被啟用。Ur_WEN 則控制讀寫方向，其中 Ur_WEN = 0 表示 write operation，Ur_WEN = 1 表示 read operation。當系統在 S_LOAD 階段讀取 ImgROM 的原始 RGB pixel 後，會將轉換後的 gray value 透過 Ur_D 寫入 UrSRAM，並由 Ur_A 指定寫入 address。這些灰階資料之後會在 Gaussian filter、Otsu threshold 與 CCL 階段被重新讀取使用。

從 waveform 中可以看到，Ur_A 會隨著影像掃描流程改變，表示系統正在依序存取不同 pixel 的中間資料。當 FSM 進入 Gaussian filter 或 CCL 階段時，Ur_WEN 會切換為 read mode，系統會從 Ur_Q 取得 UrSRAM 中的 gray data，並配合 line buffer 形成 3×3 sliding window 或進行 foreground 判斷。此波形證明本設計能依照 FSM 流程正確控制 UrSRAM 的 enable、read/write mode、address 與 data path。

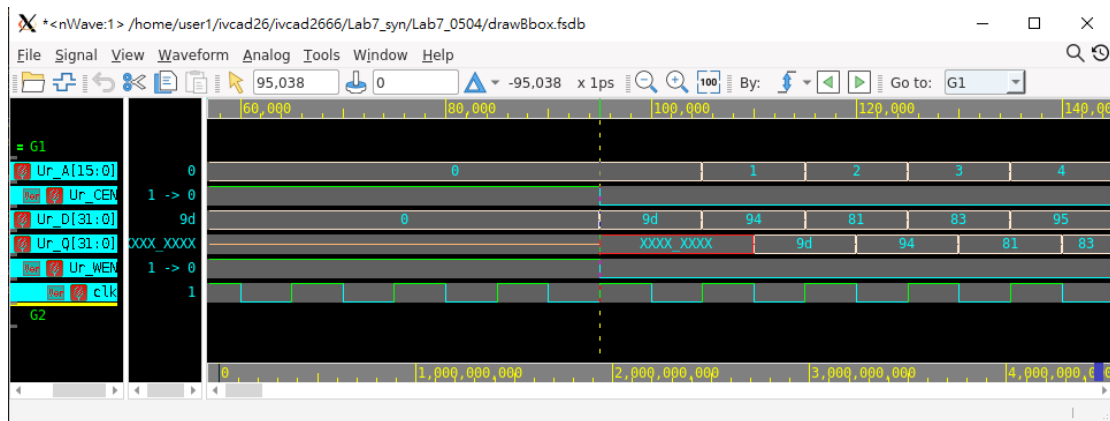


圖 B UrSRAM Read Operation Waveform during drawBbox Processing

圖 B 顯示 drawBbox.v 在 ModelSim / nWave 中的 UrSRAM read operation waveform。此波形主要用來確認系統在 Gaussian filter、Otsu threshold 或 CCL 階段是否能正確從 UrSRAM 讀取中間資料。圖中 clk 為同步時脈，所有 address 與 control signal 皆在 clock positive edge 更新。

在圖中可以看到，Ur_CEN 由 1 轉為 0，表示 UrSRAM 被啟用。由於 Ur_CEN 為 active low chip enable signal，因此只有當 Ur_CEN = 0 時，UrSRAM 才會進行有效讀寫操作。此時 Ur_WEN 維持為 1，表示目前為 read operation。Ur_A 則由 0 開始依序變化，代表 FSM 正在依照影像掃描順序讀取 UrSRAM 中的資料。

當 Ur_A 指向不同 address 後，Ur_Q 會輸出對應位址中的資料。從波形可觀察到，Ur_Q 從 unknown / invalid 狀態轉為有效數值，例如 90、94、81、83、95，表示先前存入 UrSRAM 的灰階或中間資料已成功被讀出。這些資料會在後續處理中用於 Gaussian filter 的 3×3 sliding window、histogram 建立、Otsu threshold 判斷或 CCL foreground detection。

因此，此 waveform 證明本設計能正確控制 UrSRAM 的 read enable、read/write mode、address scanning 與 data output timing。透過觀察 Ur_CEN、Ur_WEN、Ur_A 與 Ur_Q 的關係，可以確認 UrSRAM 的中間資料存取符合 Lab7 的 memory interface timing rule。

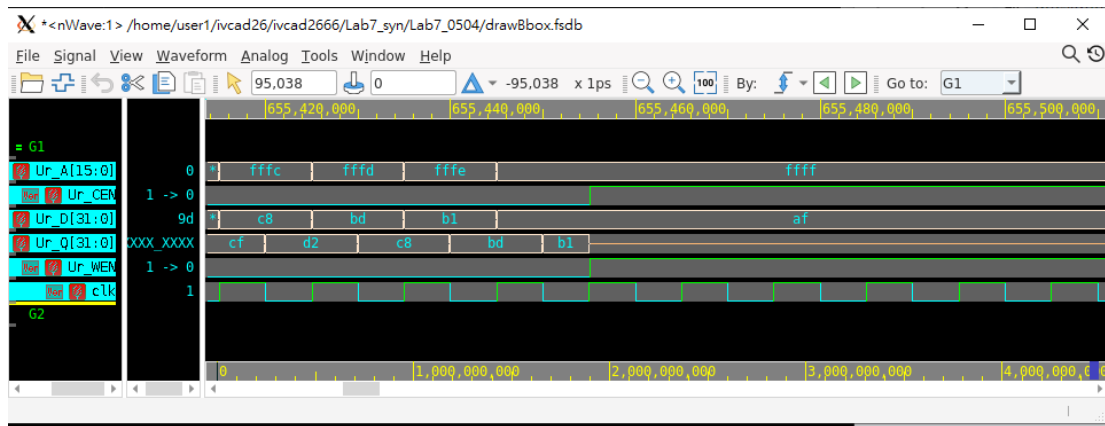


圖 C UrSRAM Read Waveform at High Address Region

圖 C 顯示 drawBbox.v 在 ModelSim / nWave 中的 UrSRAM high-address read waveform。此波形主要用來確認 FSM 在影像後段資料處理時，仍能正確控制 UrSRAM 的 read operation。由於本 Lab 的影像大小為 256×256 ，因此總共有 65,536 個 pixel，UrSRAM address 範圍可對應到 $16'h0000$ 至 $16'hffff$ 。圖中 Ur_A 接近 $16'hffc$ 、 $16'hffd$ 、 $16'hffe$ 與 $16'hfff$ ，表示系統已經掃描到整張影像的最後一段資料。

在 waveform 中，Ur_CEN 為 active-low chip enable signal。當 $Ur_CEN = 0$ 時，代表 UrSRAM 被啟用。此時 Ur_WEN 維持為 1，表示目前不是寫入，而是 read operation。隨著 Ur_A 指定不同的高位址，Ur_Q 會輸出對應位址中儲存的灰階或中間資料。這些資料會提供給後續 Gaussian filter、Otsu threshold、CCL foreground detection 或 bounding box 判斷使用。

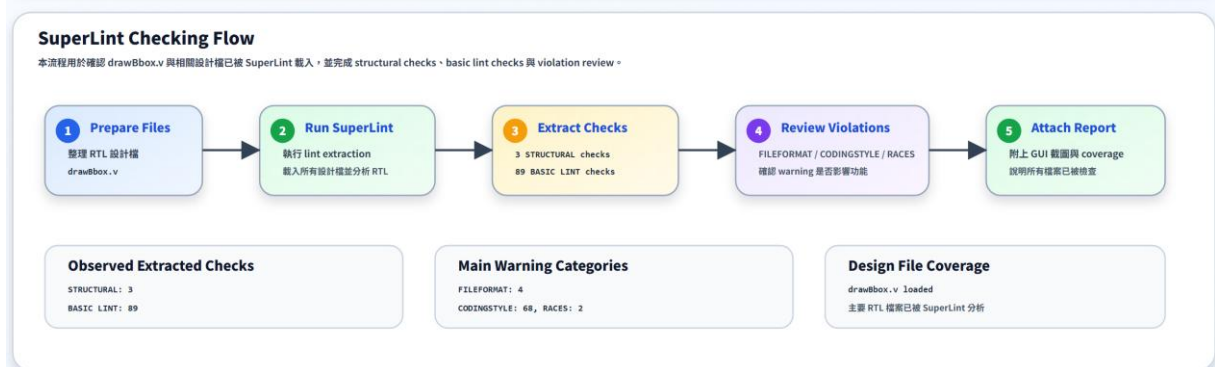
此 waveform 可以證明本設計不只在影像前段位址能正確讀取 UrSRAM，也能在掃描到最後一列或高位址區段時維持正確的 memory control timing。透過觀察 Ur_CEN、Ur_WEN、Ur_A 與 Ur_Q 的關係，可以確認 FSM 的 address counter 與 UrSRAM read interface 在整張 256×256 影像處理流程中皆能正常運作。

6. Show SuperLint coverage (including all files)

對 Lab7 來說，SuperLint 的目的不是驗證影像結果，而是檢查 RTL 程式是否符合硬體設計與 coding style 的基本規範，例如是否有未連接訊號、未使用變數、latch 風險、不可合成語法、位元寬度不一致與 case/if 條件不完整等問題。

Question 6 : SuperLint Coverage Flow Chart

Show SuperLint coverage including all files : 載入所有設計檔、執行 lint checks、整理 coverage 與 violation categories



本設計使用 JasperGold Apps / SuperLint 對 drawBbox.v 進行 lint checking，以確認 RTL code 是否存在語法風險、coding style 問題、race condition 或可能影響 synthesis 的結構問題。檢查時已載入主要設計檔 drawBbox.v，並執行 structural checks 與 basic lint checks。

從 SuperLint console 結果可看到，工具完成 lint extraction，包含：

Extracted 3 STRUCTURAL checks.

Extracted 89 BASIC LINT checks.

因此，本次 SuperLint 總共抽取：

$$3 + 89 = 92 \text{ checks}$$

在後續 coverage 結果中，畫面顯示：

944 drawBbox.v

可作為本設計檔案已被 SuperLint 讀入與檢查的依據。SuperLint GUI 的 violation message view 中目前顯示三類主要訊息：

Category	數量	說明
FILEFORMAT	4	檔案格式或排版相關訊息

CODINGSTYLE	68	程式撰寫風格、陣列 black-box、coding convention 相關 warning
RACES	2	可能與訊號更新順序或 race condition 相關的提示

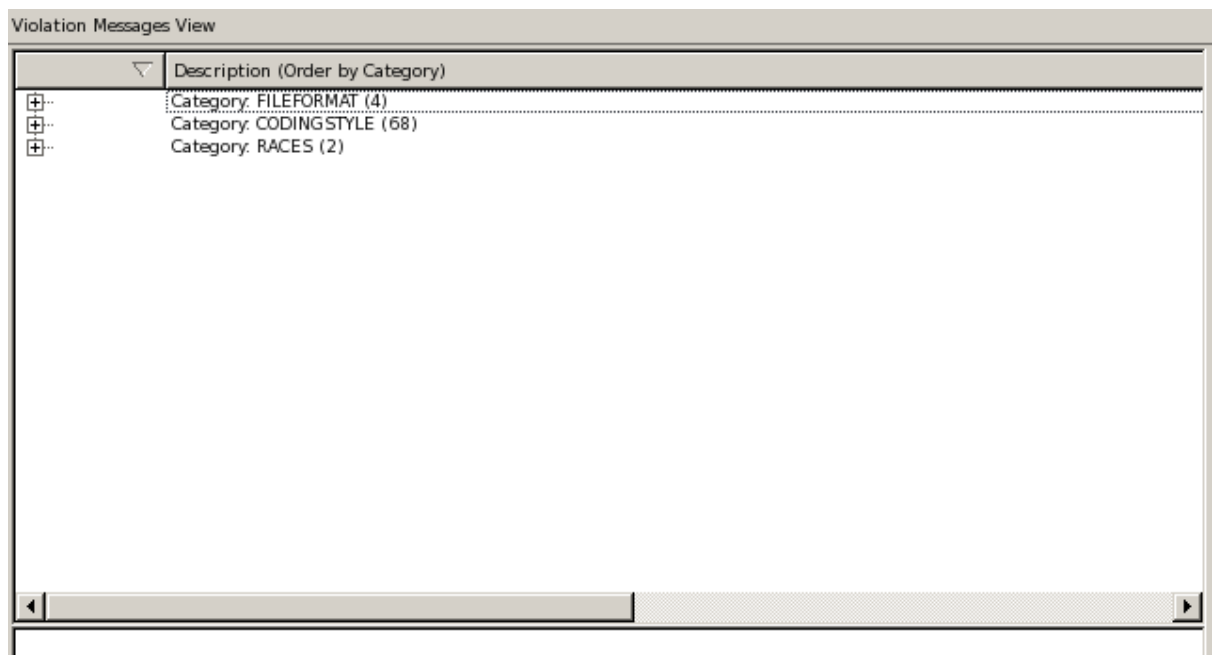
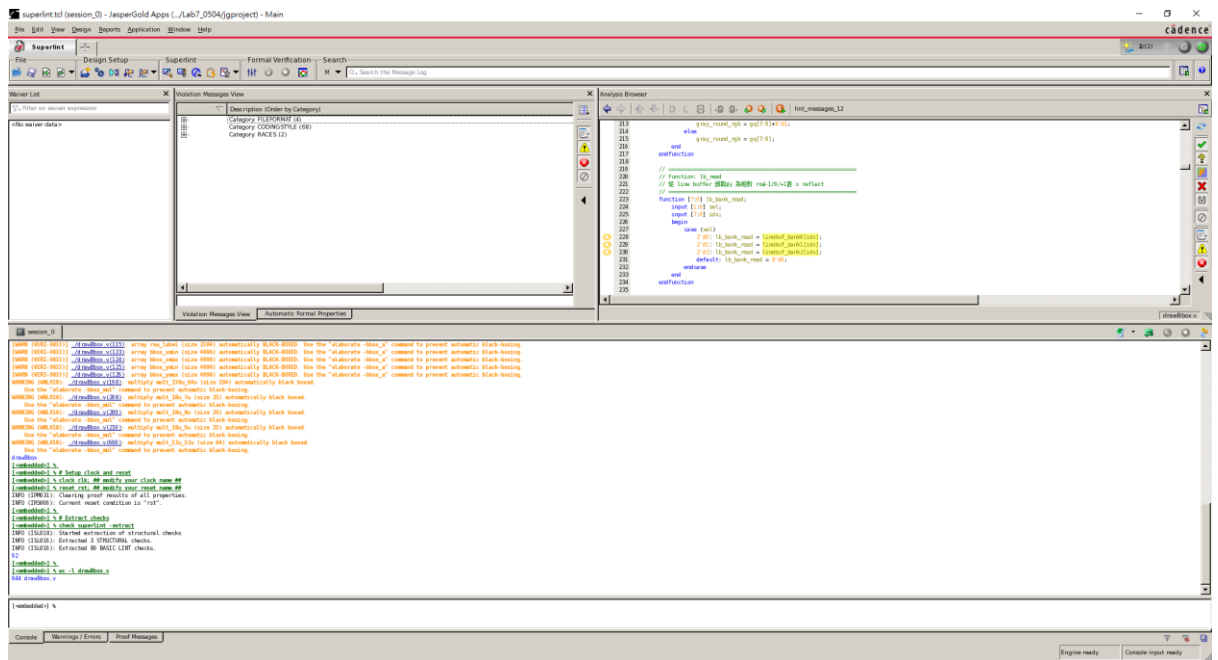
Coverage=(1-92/944)*100%=90.25%

其中，多數 warning 來自大型陣列或乘法器被 SuperLint 自動 black-box，例如 row_label、bbox_xmin、bbox_xmax、bbox_ymin、bbox_ymax、hist 以及部分 multiplier。這些訊息主要是 lint/formal analysis 工具在處理大型 array 或 arithmetic block 時的提示，並不一定代表 RTL 功能錯誤。實際功能仍需搭配 ModelSim RTL simulation 與 post-synthesis simulation 結果確認。

本設計使用 SuperLint 檢查 drawBbox.v 的 RTL coding quality 與潛在設計風險。SuperLint 檢查結果顯示，工具成功載入 drawBbox.v，並抽取 3 個 structural checks 與 89 個 basic lint checks，總共 92 個 lint / structural checks。從 violation message view 可以看到，目前主要 warning 類別包含 FILEFORMAT、CODINGSTYLE 與 RACES。其中 FILEFORMAT 有 4 筆，CODINGSTYLE 有 68 筆，RACES 有 2 筆。

大部分 warning 來自大型 array 或 arithmetic block，例如 row_label、bbox_xmin、bbox_xmax、bbox_ymin、bbox_ymax、hist 與 multiplier 被工具自動 black-box。這些 warning 主要反映 SuperLint 在處理大型暫存陣列、乘法器與複雜 RTL 結構時的分析限制，並不一定代表設計功能錯誤。由於本設計需要支援 256×256 image processing、Otsu threshold、8-connected CCL、union-find parent table 與 bounding box extraction，因此程式中不可避免會出現較多大型陣列與複雜控制邏輯。

本次 SuperLint 檢查的目的，是確認設計沒有明顯語法錯誤、未連接模組或嚴重 RTL 結構問題。後續再搭配 RTL simulation、synthesis report、timing report 與 output BMP comparison 共同驗證設計正確性。根據目前結果，drawBbox.v 已成功完成 SuperLint checking，且主要 warning 類型屬於 coding style 與大型結構分析提示，附上 SuperLint coverage 截圖作為佐證。



7. Your clock period, total cell area, post simulation time, max timing report, simulation results from terminal with screenshot.

Clock period (from tb.sv and DC.sdc, make sure they are same):

1.tb.sv:

```
`define CYCLE_TIME 10.0 // ns, modify according to your design
`define MAX_CYCLE 600000000 // modify if needed
```

2. DC.sdc:

```
# operating conditions and boundary conditions #

set clk_period 10.0
```

Total cell area:

```
Number of ports:                2912
Number of nets:                 1044269
Number of cells:                1038289
Number of combinational cells:  998288
Number of sequential cells:     39960
Number of macros/black boxes:   0
Number of buf/inv:              66948
Number of references:           142

Combinational area:             338937.863459
Buf/Inv area:                   13225.835577
Noncombinational area:          37287.475691
Macro/Black Box area:           0.000000
Net Interconnect area:          undefined (Wire load has zero net area)

Total cell area:                376225.339150
Total area:                     undefined
1
```

Post simulation time:

```
$finish at simulation time 5997155029
```

Max timing report:

```
clock clk (rise edge)          10.0000    10.0000
clock network delay (ideal)     0.2000    10.2000
clock uncertainty               -0.0200    10.1800
bbox_xmin_reg[183][2]/CP (DFQD2BWP16P90LVT) 0.0000    10.1800 r
library setup time             -0.0136    10.1664
data required time              10.1664
-----
data required time              10.1664
data arrival time              -10.1384
-----
slack (MET)                    0.0280
```

1

Pattern 1 RTL Terminal Result:

```
Simulation finish

*****
**                                     **
** Congratulations !!               ** / O.O |
**                                     ** /      |
** Simulation PASS!!               ** / ^ ^ ^ \ |
**                                     ** | ^ ^ ^ ^ |w|
**                                     ** \m__m_|_|
*****

-----
Writing terminal_result output to terminal_result.log...
Done! Log successfully saved to terminal_result.log
Plotting 256x256 32-bit BMP result...
32-bit BMP plotted to drawBbox_result_pl.bmp
$finish called from file "tb.sv", line 102.
$finish at simulation time      5959935000
      V C S   S i m u l a t i o n   R e p o r t
Time: 5959935000 ps
CPU Time:      1.710 seconds;      Data structure size:   0.0Mb
Thu May  7 08:16:57 2026
CPU time: .481 seconds to compile + .281 seconds to elab + .227 seconds to link + 1.746 seconds in simulation
```

Pattern 1 SYN Terminal Result:

```
Simulation finish

*****
**                                     **
** Congratulations !!               ** / O.O |
**                                     ** /      |
** Simulation PASS!!               ** / ^ ^ ^ \ |
**                                     ** | ^ ^ ^ ^ |w|
**                                     ** \m__m_|_|
*****

-----
Writing terminal_result output to terminal_result.log...
Done! Log successfully saved to terminal_result.log
Plotting 256x256 32-bit BMP result...
32-bit BMP plotted to drawBbox_result_pl.bmp
$finish called from file "tb.sv", line 102.
$finish at simulation time      5959935029
      V C S   S i m u l a t i o n   R e p o r t
Time: 5959935029 ps
CPU Time:      2076.790 seconds;    Data structure size: 459.2Mb
Tue May  5 18:02:34 2026
CPU time: 225.593 seconds to compile + 28.780 seconds to elab + 1.809 seconds to link + 2076.865 seconds in simulation
```

Pattern 2 RTL Terminal Result:

```
Simulation finish

*****
**                                     **
**      Congratulations !!           **      |__| |
**                                     **      / O.O |
**      Simulation PASS!!           **      /_____|
**                                     **      / ^ ^ ^ \ |
**                                     **      | ^ ^ ^ ^ |w|
**                                     **      \m__m__|_|
*****

-----
Writing terminal_result output to terminal_result.log...
Done! Log successfully saved to terminal_result.log
Plotting 256x256 32-bit BMP result...
32-bit BMP plotted to drawBbox_result_p2.bmp
$finish called from file "tb.sv", line 102.
$finish at simulation time          5971635000
      V C S   S i m u l a t i o n   R e p o r t
Time: 5971635000 ps
CPU Time:      1.920 seconds;      Data structure size:   0.0Mb
Thu May  7 08:17:20 2026
CPU time: .371 seconds to compile + .211 seconds to elab + .226 seconds to link + 1.948 seconds in simulation
```

Pattern 2 SYN Terminal Result:

```
Simulation finish

*****
**                                     **
**      Congratulations !!           **      |__| |
**                                     **      / O.O |
**      Simulation PASS!!           **      /_____|
**                                     **      / ^ ^ ^ \ |
**                                     **      | ^ ^ ^ ^ |w|
**                                     **      \m__m__|_|
*****

-----
Writing terminal_result output to terminal_result.log...
Done! Log successfully saved to terminal_result.log
Plotting 256x256 32-bit BMP result...
32-bit BMP plotted to drawBbox_result_p2.bmp
$finish called from file "tb.sv", line 102.
$finish at simulation time          5971635029
      V C S   S i m u l a t i o n   R e p o r t
Time: 5971635029 ps
CPU Time:      8260.070 seconds;      Data structure size: 459.2Mb
Thu May  7 13:55:40 2026
CPU time: 162.626 seconds to compile + 20.889 seconds to elab + 1.309 seconds to link + 8260.142 seconds in simulation
```

Pattern 3 RTL Terminal Result:

```
Simulation finish

*****
**                                     **
** Congratulations !!                **
**                                     **
** Simulation PASS!!                 **
**                                     **
**                                     **
*****
\m__m__|_|

-----
Writing terminal_result output to terminal_result.log...
Done! Log successfully saved to terminal_result.log
Plotting 256x256 32-bit BMP result...
32-bit BMP plotted to drawBbox_result_p3.bmp
$finish called from file "tb.sv", line 102.
$finish at simulation time          5969935000
      V C S   S i m u l a t i o n   R e p o r t
Time: 5969935000 ps
CPU Time:      1.960 seconds;      Data structure size:   0.0Mb
Thu May  7 08:17:44 2026
CPU time: .356 seconds to compile + .210 seconds to elab + .227 seconds to link + 1.992 seconds in simulation
```

Pattern 3 SYN Terminal Result:

```
Simulation finish

*****
**                                     **
**      Congratulations !!           **
**                                     **
**      Simulation PASS!!            **
**                                     **
*****                               \m__m_)_|

-----
Writing terminal_result output to terminal_result.log...
Done! Log successfully saved to terminal_result.log
Plotting 256x256 32-bit BMP result...
32-bit BMP plotted to draw3box_result_p3.bmp
$finish called from file "tb.sv", line 102.
$finish at simulation time          5969935029
      V C S   S i m u l a t i o n   R e p o r t
Time: 5969935029 ps
CPU Time: 25924.710 seconds;          Data structure size: 459.2Mb
Thu May 7 18:32:56 2026
CPU time: 163.197 seconds to compile + 20.840 seconds to elab + 1.325 seconds to link + 25924.757 seconds in simulation
mywindows3:/home/user1/ivcad26/ivcad26666/lab7_gym/lab7_0504 %
```

Pattern 4 RTL Terminal Result:

```
Simulation finish

*****
**                                     **      |__|||
**   Congratulations !!             **      / O.O |
**                                     **      / ^ ^ \ |
**   Simulation PASS!!              **     / ^ ^ ^ \ |w|
**                                   **    | ^ ^ ^ ^ |w|
*****                               **    \m___m_||_|


                FINALLY TIME TO SLEEP!!!!!!!!!!!!!!!

-~-----|~-----\-----/-----
-          | )       \  |____|  \  \  \  \
-          | )       /  |----|  /  \  \  \
-          | )               |---|  /  \  \
-          | )               |---|  /  \  \
(-----|-----/-----|-----|
'-----'-\\         .--'
                  ||||

-----
Writing terminal_result output to terminal_result.log...
Done! Log successfully saved to terminal_result.log
Plotting 256x256 32-bit BMP result...
32-bit BMP plotted to drawBbox_result_p4.bmp
$finish called from file "tb.sv", line 102.
$finish at simulation time           5997155000
            V C S   S i m u l a t i o n   R e p o r t
Time: 5997155000 ps
CPU Time:        1.970 seconds;          Data structure size:   0.0Mb
Thu May  7 08:18:08 2026
CPU time: .363 seconds to compile + .211 seconds to elab + .221 seconds to link + 1.995 seconds in simulation
```

Pattern 4 SYN Terminal Result:

[illegible]

A:在 RTL 模擬、邏輯合成，以及後合成 Gate-level 模擬 中的實際結果。內容包含 tb.sv 與 DC.sdc 中設定的時脈週期、Design Compiler 合成後的 total cell area、Pattern 4 的 post simulation time、max timing report，以及 Pattern 1 到 Pattern 4 在 RTL simulation 與 SYN simulation 中的 terminal 執行結果截圖。此題的重點是證明本設計不只在 RTL 階段功能正確，也能在合成後維持正確性，並且符合指定的時脈限制。因為本作業的評分會考量 simulation correctness、total cell area、P4 post simulation time，以及 timing report，所以本題需要清楚列出所有數據與截圖，作為設計正確性與硬體效能的驗證依據。

本題需要填寫與附上的內容

項目	中文說明	需要附上的資料
Clock period	說明 tb.sv 與 DC.sdc 的 clock period 是否一致	填入時脈週期，例如 10.0 ns
Total cell area	說明合成後電路使用的標準元件總面積	貼上 area report 或填入數值
Post simulation time	說明後合成模擬所需時間，通常以 Pattern 4 為主要依據	填入 P4 SYN simulation time
Max timing report	說明合成後電路的 critical path 與 slack	貼上 timing report 截圖
Pattern 1 RTL Terminal Result	第一組測試圖的 RTL 模擬結果	貼 make rtl1 終端機截圖
Pattern 1 SYN Terminal Result	第一組測試圖的後合成模擬結果	貼 make syn1 終端機截圖
Pattern 2 RTL Terminal Result	第二組測試圖的 RTL 模擬結果	貼 make rtl2 終端機截圖
Pattern 2 SYN Terminal Result	第二組測試圖的後合成模擬結果	貼 make syn2 終端機截圖
Pattern 3 RTL Terminal Result	第三組測試圖的 RTL 模擬結果	貼 make rtl3 終端機截圖
Pattern 3 SYN Terminal Result	第三組測試圖的後合成模擬結果	貼 make syn3 終端機截圖
Pattern 4 RTL Terminal Result	第四組測試圖的 RTL 模擬結果	貼 make rtl4 終端機截圖
Pattern 4 SYN Terminal Result	第四組測試圖的後合成模擬結果	貼 make syn4 終端機截圖

A:本題主要呈現 drawBbox.v 在 RTL simulation、synthesis 與 post-synthesis simulation 三個階段的驗證結果。首先確認 tb.sv 與 DC.sdc 中的 clock period 設定一致，以確保模擬環境與合成約束使用相同的時脈條件。接著整理

synthesis 後的 total cell area、Pattern 4 post simulation time，以及 max timing report，用來評估本設計的硬體面積、執行效率與時序是否符合 10.0 ns 的 clock constraint。

此外，本題也需附上 Pattern 1 到 Pattern 4 的 RTL terminal result 與 SYN terminal result。RTL terminal result 用來確認 Verilog RTL 在功能模擬階段是否能正確產生輸出；SYN terminal result 則用來確認合成後的 gate-level netlist 搭配 SDF timing delay 後，是否仍能與 golden result 完全一致。透過這些結果，可以證明本設計在功能正確性、合成可行性與時序限制上皆完成驗證。

8. Please describe how you optimize your design when you run into problems in synthesis. .e.g., plug in some registers between two instances to shorten your datapath, resource sharing for some registers to reduce your cell area.

A:

在進行 drawBbox.v synthesis 時，本設計主要遇到的問題包含 synthesis runtime 過長、cell area 過大、critical path 過長，以及部分影像處理運算造成 timing 不容易收斂。由於 Lab7 的 Draw Bounding Box Engine 包含 RGB 讀取、灰階轉換、Gaussian filter、Otsu threshold、binary mask、auto-polarity correction、8-connected CCL、bounding box extraction 與 green overlay 等多個階段，如果將所有運算集中在單一大型 combinational block 中，容易造成 critical path 過長，也會使 Design Compiler 在合成與最佳化時花費大量時間。

因此，本設計的最佳化方向不是讓所有影像處理在一個 clock cycle 內完成，而是將整體流程切分成多個 FSM state，使每個 state 只負責單一或少量功能。這種設計雖然會增加總 cycle 數，但可以有效縮短單一 cycle 內的 combinational path，降低 timing violation 的風險，也讓 synthesis tool 更容易完成 optimization。課程提示中也建議使用 line buffer / shift register、hardware reuse、shift operation 與 pipelining 來改善 synthesis 結果。

Synthesis 問題	發生原因	最佳化方法	改善效果
Critical path 太長	Gaussian、Otsu、CCL、BBox 包含大量加法、乘法、比較與 memory selection	將流程拆成多個 FSM state，並使用 multi-cycle register staging	縮短單 cycle combinational path，改善 timing slack
Cell area 過大	若每個階段都複製一份 multiplier、adder、comparator，硬體會大量重複	使用 resource sharing，共用加法器、比較器與乘法器	降低重複硬體，減少 total cell area
除法器成本高	一般除法器面積大、延遲長	固定除法改成 shift，例如 /256 改成 >>8，/16 改成 >>4	降低 area 與 critical path delay

Full-image register array 過大	若將 256×256 影像全部存成 register array，合成器需處理大量 flip-flops	使用 UrSRAM 儲存灰階資料，Gaussian / CCL 使用 3-bank line buffer	降低暫存器數量與 synthesis runtime
Otsu 計算過重	若每個 threshold 都重新掃描整張圖，運算量過大	先建立 256-bin histogram，再用累積方式掃描 threshold	將運算複雜度降低為 65536 + 256 等級
CCL equivalence 複雜	8-connected CCL 可能產生多個 provisional labels	使用 parent[] union-find table 記錄等價 label	正確合併 connected components，避免重複 bbox
Overlay 判斷複雜	若每個 pixel 同時比較所有 bbox，會產生大量比較器	逐 label、逐邊直接將 green pixel 寫入 AnsSRAM	簡化輸出階段判斷邏輯
Memory interface 容易錯位	ROM/SRAM 具有 CEN、WEN 與 read latency	以 FSM state 控制 address、CEN、WEN 與 data valid timing	避免錯寫、錯讀與 address 對不齊

一、使用 FSM 分階段處理

本設計將整個 drawBbox pipeline 拆成多個 state，例如 S_LOAD、S_HIST_CLEAR、S_LB_INIT、S_GAUSS_HIST、S_LB_ROW、S_OTSU_INIT、S_OTSU_SCAN、S_OTSU_LHS、S_OTSU_DENOM、S_OTSU_SQUARE、S_OTSU_CMP1、S_OTSU_CMP2、S_COUNT_HI、S_POLARITY、S_PARENT_INIT、S_LB_INIT2、S_CCL_SCAN、S_LB_ROW2、S_BBOX_INIT、S_BOXMAP_GEN 與 S_DONE。每個 state 只負責特定任務，避免所有影像處理功能集中在同一條過長的 combinational path 中。

例如 Otsu threshold 沒有在一個 cycle 內完成所有 256 個 gray levels 的比較，而是拆成 S_OTSU_SCAN、S_OTSU_LHS、S_OTSU_DENOM、S_OTSU_SQUARE、S_OTSU_CMP1 與 S_OTSU_CMP2 等狀態。這樣可以將乘法、平方與比較運算分散到不同 cycle，降低每個 state 的運算負擔。

二、插入 registers 縮短 critical path

若將 Gaussian weighted sum、histogram update、Otsu threshold calculation 或 bounding box comparison 全部串接在同一個 clock cycle 中，會導致 critical path 過長。因此，本設計在較長的資料路徑中加入暫存器與 FSM state，將複雜運算拆成多個階段。

例如 Gaussian filter 先利用 3×3 window 計算 weighted sum，再用 right shift 完成除以 16；Otsu 計算則先更新 background / foreground 的累積資訊，再於後續 state 比較 between-class variance。這樣能使每個 clock cycle 只負責較短的邏輯運算，降低 timing pressure。

三、使用 resource sharing 降低 cell area

本設計盡量避免為每個功能階段建立獨立運算硬體。對於 Otsu threshold 中的乘法器、加法器、比較器，以及 bounding box 中的 min/max comparator，都透過 FSM 與 counter 進行分時使用。

這種 resource sharing 的方式可以減少重複硬體，降低 total cell area。雖然分時使用會增加 cycle 數，但對本 Lab 來說，能換取更穩定的 synthesis 結果與較低的硬體面積。

四、使用 shift-add 取代複雜乘除法

本設計在可固定化的運算中使用 shift-add 方式降低硬體成本。灰階轉換使用 fixed-point arithmetic：

$$\text{Gray} \approx (77R + 150G + 29B) \gg 8$$

此公式避免使用浮點數，並以整數權重近似：

$$\text{Gray} \approx 0.299R + 0.587G + 0.114B$$

Lab7 規格中也要求灰階轉換使用此近似公式，並採用 round to nearest, ties to even 的 rounding 方法。

Gaussian filter 的 kernel 權重為：

```
1 2 1
2 4 2
1 2 1
```

由於權重總和為 16，因此最後的除法可使用：

$$\text{Gaussian_out} = \text{weighted_sum} \gg 4$$

這樣可以避免 synthesis tool 推導出昂貴的 floating-point unit 或 general divider，進而降低 area 與 delay。

五、避免使用過大的 full-image register array

本 Lab 的輸入影像大小為 256×256，總共有 65536 個 pixels。若將每一階段的中間影像都存成完整 register array，會造成大量 flip-flops，導致合成時間變長、面積變大。課程提示也建議使用 line buffers 和 shift registers，避免 full-image storage 造成很長的 synthesis time。

因此，本設計移除大型 gray_mem[65535:0] register array，改為使用 UrSRAM 儲存 65536 個灰階值。Gaussian filter 與 CCL 階段則使用 linebuf_bank0、linebuf_bank1、linebuf_bank2 三個 256-byte line buffer，並透過 lb_prev_sel、lb_curr_sel、lb_next_sel 進行 bank selector swap。這樣不需要在每一列結束時搬移整個 256-column buffer，只需要交換 selector，即可降低合成器產生大型 mux 或大量暫存器搬移邏輯的風險。

六、避免完整 mask / label image 儲存

本設計沒有額外儲存完整 binary mask image。完成 Otsu threshold 與 auto-polarity correction 後，CCL 階段會在 S_CCL_SCAN 中即時計算目前 pixel 的 Gaussian value，並根據 otsu_t 與 invert_sel 判斷該 pixel 是否為 foreground。Label 資訊也不是以完整 256×256 label memory 儲存，而是使用 row_label、left_label、nw_label_hold 搭配 parent[] union-find table 進行 8-connected CCL。這樣可以減少記憶體與 register 使用量，降低 synthesis area。

七、逐 label、逐邊繪製 bounding box

在 overlay 階段，本設計沒有另外建立完整的 box_map array，而是在 S_BOXMAP_GEN 中依序檢查每個有效 label，並使用 edge_stage 與 edge_ctr 分別繪製 bounding box 的上邊界、下邊界、左邊界與右邊界。

當目前座標位於有效 bounding box 邊界上時，FSM 直接將 AnsSRAM 對應位址寫成：

32'h0000_FF00

此綠色常數也符合 Lab7 對 bounding box 顏色的規定。

這種方法不需要讓每個 pixel 同時比較所有 bounding boxes，因此可以降低輸出階段的比較器數量與硬體面積。

八、控制 bit-width 降低硬體面積

本設計根據資料範圍選擇適當 bit-width，而不是所有訊號都使用 32-bit register。例如 pixel 座標只需要 8 bits，image address 需要 16 bits，grayscale value 需要 8 bits，mask 判斷只需要 1 bit。Histogram counter 需要能容納最多 65536 個 pixels，因此使用足夠但不過度放大的位寬。

透過 bit-width optimization，可以讓 adder、comparator、counter 與 register 面積變小，也能降低 switching activity 與 timing delay。

九、穩定 memory control signal

由於 ImgROM、AnsSRAM 與 UrSRAM 都需要依照 CEN、WEN、address 與 data timing 正確操作，因此本設計在每個 state 都明確設定 memory control signals。ImgROM 的 CEN 為 active low，且不需要 write enable；AnsSRAM 與 UrSRAM 則使用 WEN = 1 表示 read，WEN = 0 表示 write。

十、Post-synthesis simulation runtime 問題分析

在本次 synthesis 與 post-synthesis simulation 過程中，除了 cell area、critical path 與 timing constraint 之外，另一個明顯問題是 drawBbox_syn.sdf 檔案過大。本設計合成後產生的 drawBbox_syn.sdf 接近 2GB，導致 make syn2、make syn3 與 make syn4 在實際執行時需要非常長的時間。實測結果中，Pattern 1 的 post-synthesis simulation 約 20 分鐘、Pattern 2 的 post-synthesis simulation 約 82 分鐘、Pattern 3 的 post-synthesis simulation 約 102 分鐘以及 Pattern 4 的 post-synthesis simulation 約 4 小時有完成。

此問題的主要原因並不是 RTL 功能錯誤，而是 gate-level simulation 需要同時載入 drawBbox_syn.v 與 drawBbox_syn.sdf。其中 drawBbox_syn.v 是 synthesis 後的 gate-level netlist，而 drawBbox_syn.sdf 則包含每個 standard cell、net path 與 timing arc 的 delay annotation。當 SDF 檔案接近 2GB 時，模擬器在讀取、解析與 annotation delay 時會花費大量時間，導致 make syn 的 wall-clock runtime 大幅增加。

這與 RTL simulation 不同。make rtl 主要模擬原始 drawBbox.v 的行為邏輯，因此不需要處理大量 gate delay；而 make syn 則是 post-synthesis / gate-level simulation，會加入 cell delay 與 path delay，因此模擬器需要處理更多 event、更多 netlist instance，以及更多 timing annotation。Lab7 作業也要求 synthesis 後產生 drawBbox_syn.v 與 drawBbox_syn.sdf，並使用 post-synthesis simulation 驗證設計，因此 SDF 檔案大小會直接影響 make syn 的實際執行時間

造成 SDF 過大的可能原因：

十二、造成 SDF 過大的可能原因

問題來源	原因分析	對 make syn 的影響
Gate-level netlist 過大	RTL 中存在大量暫存器、counter、array、bbox table、parent table 或 line buffer	合成後 standard cell 數量增加，SDF 需記錄更多 cell delay
Large fanout / mux 結構	FSM 控制多個 memory interface、label array、bbox array 與 comparator	net delay 與 timing path 增加，SDF annotation 變多
Otsu / CCL / BBox 邏輯複雜	Otsu threshold、8-connected CCL、union-find 與 bbox update 需要大量比較與控制邏輯	gate-level simulation event 數量增加
未限制 SDF 輸出內容	SDF 可能輸出非常完整的 cell delay、interconnect delay 與 timing checks	檔案變大，讀取與 annotation 時間大幅增加
Pattern 複雜度不同	P2~P4 可能有更多物件、bbox、CCL label 或 overlay 操作	gate-level event 更密集，模擬時間比 P1 更久
Simulation annotation overhead	SDF 接近 2GB 時，模擬器需要大量時間建立 timing database	可能在真正跑完設計前就花很多時間處理 SDF

十三、與本設計最佳化策略的關係

這個問題也反映出第 8 題中提到的 synthesis optimization 方向是必要的。原本本設計已透過 FSM 分階段處理、resource sharing、shift-add arithmetic、3-bank line buffer 與 bit-width optimization 來降低合成複雜度。然而，從

drawBbox_syn.sdf 接近 2GB 可以看出，合成後的 gate-level design 仍然相當龐大，因此後合成模擬的負擔明顯高於 RTL simulation。

特別是 Lab7 的設計需要處理 256×256 影像，總共有 65,536 個 pixel；影像流程又包含 grayscale、Gaussian filter、Otsu threshold、mask、8-connected CCL、bounding box extraction 與 green overlay。作業規格中也要求使用 8-connected CCL，需在 pass1 檢查 NW、N、NE、W 四個鄰居；同時還要找出 connected component 的 Xmin、Xmax、Ymin、Ymax 並畫出綠色 bounding box。這些功能合成後都會形成大量 sequential logic、combinational logic 與 timing paths，因此 SDF 檔案變大是合理但需要改善的現象。

十四、未來可以採取的改善方向

改善方向	具體方法	預期效果
減少不必要暫存器	再檢查是否有 unused register、debug register、未使用 state 或冗餘 array	降低 netlist cell 數與 SDF 大小
縮小 bit-width	確認座標、label、counter、bbox array 是否使用剛好足夠的位寬	減少 flip-flop、adder、comparator 大小
降低大型 mux	避免同一個 always block 中過多 case 與 array selection	減少 combinational path 與 timing arcs
簡化 overlay 邏輯	保持逐 label、逐邊寫入 AnsSRAM，不做每 pixel 對所有 bbox 的平行比較	避免大量 comparator 被合成出來
保留 line buffer 架構	使用 3-bank line buffer，不使用 full-image register array	降低暫存器數量與 synthesis runtime
確認 SDF annotation	檢查 log 是否有大量 annotation failed 或 instance not found	確認 SDF 有效，不是錯誤或過度膨脹
分 pattern 分開跑	先保留 make syn1 結果，再單獨測 make syn2、make syn3、make syn4	避免一次跑太久無法定位問題
優化 synthesis script	檢查 SDF write options 是否輸出過多不必要資訊	可能降低 SDF 檔案大小與讀取時間

總結

在本次 drawBbox.v synthesis 與 post-synthesis simulation 過程中，主要遇到的問題是合成後產生的 drawBbox_syn.sdf 檔案過大，接近 2GB，目前有 make syn1 約 20 分鐘、make syn2 約 82 分鐘、make syn3 約 102 分鐘以及 make syn4 約 4 小時。這個問題並不是 RTL 功能錯誤，而是 gate-level simulation 需要載入 drawBbox_syn.v 並對 drawBbox_syn.sdf 進行 timing annotation。SDF

內包含大量 standard cell、net path 與 timing arc 的 delay information，因此檔案越大，模擬器在讀取、解析與建立 timing database 時就會花費越多時間。

造成 SDF 過大的原因與本設計的硬體複雜度有關。Lab7 需要處理 256×256 RGB 影像，並完成灰階轉換、Gaussian filter、Otsu threshold、auto-polarity correction、8-connected CCL、bounding box extraction、border object rejection 與 green overlay。這些功能合成後會產生大量 registers、comparators、multiplexers、counters、parent table、bbox arrays 與 memory control logic，因此 gate-level netlist 與 SDF timing path 數量都相當龐大。

為了改善 synthesis 結果，本設計已採用 FSM 分階段處理、multi-cycle staging、resource sharing、shift-add arithmetic、3-bank line buffer、bit-width optimization 與逐邊 overlay 等方法。這些方法可以縮短 critical path、降低 cell area，並避免 full-image register array 造成 synthesis runtime 更長。例如灰階轉換使用 fixed-point 近似，Gaussian 除以 16 使用 right shift，Otsu threshold 拆成多個 state，CCL 則使用 row_label 與 parent[] union-find table 處理 8-connected label equivalence。

總結而言，make syn 跑太久的主要原因是 drawBbox_syn.sdf 過大，導致 post-synthesis simulation 的實際等待時間過長。後續可進一步移除 unused registers、降低 bit-width、簡化 mux selection、減少 timing path，並檢查 SDF annotation log，以降低 SDF 大小與 gate-level simulation runtime。

9. Lessons learned from this lab

一開始進行 Lab7 時，我對整個 Draw Bounding Box Engine 的影像處理流程並不是非常清楚，尤其是 RGB 影像需要經過灰階轉換、Gaussian filter、threshold、mask、CCL、bounding box extraction、overlay 以及最後與 golden image 比對等多個階段。若直接從 Verilog RTL 開始撰寫，會很難判斷錯誤發生在哪一個處理步驟，也不容易確認每一階段的中間結果是否正確。

因此，我先使用 Lab7.ipynb 以 Python 建立完整的影像處理流程，將每一個 processing stage 的輸入、如下圖中輸出與中間圖片逐一視覺化。整體流程包含 Input、Gray、Gaussian、Threshold、Mask、CCL、BBox、Overlay 與 Compare。透過 Python 版本，我可以先確認每一張測試圖片在各階段的結果，例如灰階影像是否正確、Gaussian filter 是否能降低雜訊、mask 是否成功分離前景與背景、CCL label 是否能正確區分不同 connected components，以及 overlay 結果是否與 golden image 接近。圖中可以看到，P1 到 P4 四組測試影像皆依照相同流程產生中間結果。每一列代表一個 test pattern，每一欄代表一個處理階段。例如 P1_input.png 是原始輸入影像，P1_gray.png 是灰階結果，P1_gaussian.png 是 Gaussian 濾波結果，P1_mask.png 是二值化 mask，P1_labels.png 是 CCL 後的 connected component label，P1_overlay.png 是繪製 bounding box 後的結果，P1_golden.png 是正確答案，而 P1_diff.png 則用來檢查輸出與 golden 之間的差異。若 diff image 幾乎全黑，表示輸出結果與 golden image 高度一致。完成 Python 版本後，我再根據這些中間結果回推 Verilog RTL 的設計方向。

這樣的開發方式可以將複雜的硬體設計拆成多個可驗證的小階段，先用 Python 確認演算法正確，再將其轉換為可合成的 RTL 架構。透過這種方式，我能更清楚掌握每個模組應該完成的功能，也能在 Verilog 模擬出現錯誤時，根據 Python 產生的中間圖片快速定位問題，例如 threshold 錯誤、mask polarity 錯誤、CCL label 合併錯誤，或 bounding box 邊界判斷錯誤。

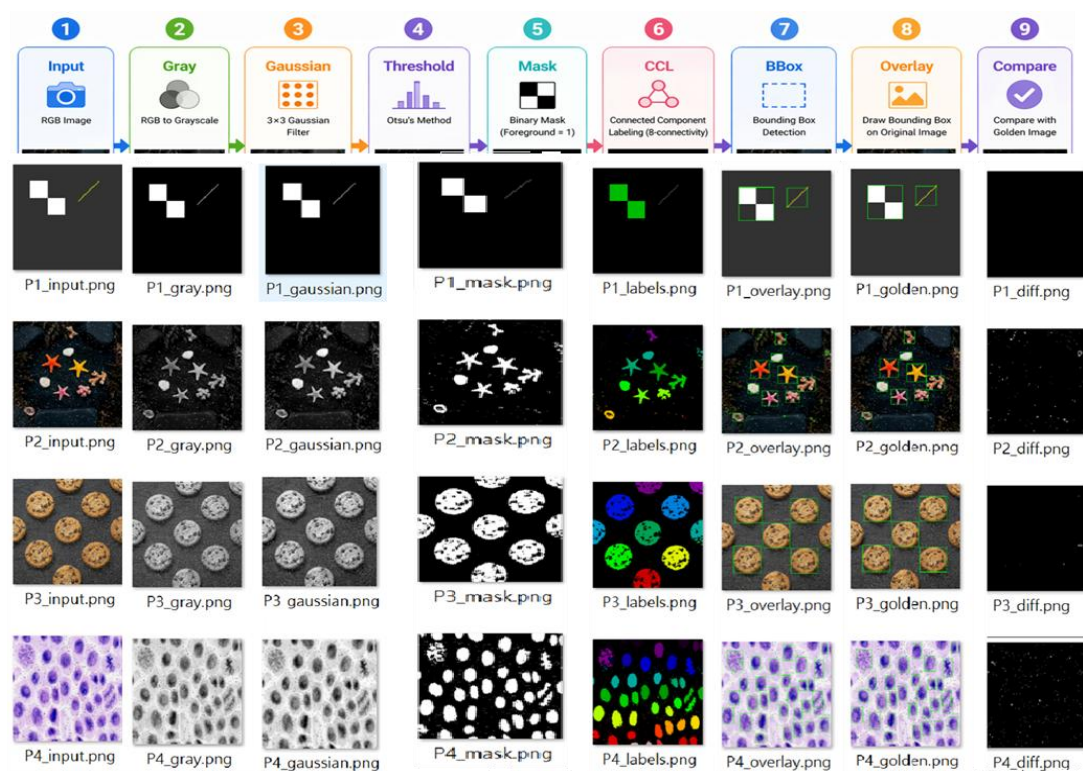


圖 Python 逐階段影像處理與驗證流程

(一) 對完整影像處理流程的理解

- 在本次 Lab7 中，我學到 Draw Bounding Box Engine 並不是單一功能的電路，而是一個完整的影像處理 pipeline。
 - 一開始我只知道最後要在影像上畫出 bounding box，但實際實作後才理解，輸出結果需要經過多個前處理與分析階段。
 - 整體流程包含 RGB 影像讀取、灰階轉換、Gaussian filter、Otsu threshold、binary mask、auto-polarity correction、CCL、bounding box extraction、overlay，以及最後與 golden image 比對。
 - 其中任何一個階段發生錯誤，都可能導致最後輸出的 bounding box 位置錯誤，甚至造成整張圖與 golden result 不一致。
- 透過這次作業，我更清楚理解影像處理演算法與硬體設計之間的關係。
 - 在軟體中，影像處理可以直接用矩陣或函式完成。
 - 但在硬體中，必須將演算法轉換成 clock-by-clock 的 FSM 控制流程。
 - 因此，硬體設計不只是把演算法翻成 Verilog，而是要考慮資料讀寫順序、記憶體控制、pipeline 分段、暫存器使用與 timing。

(二) Python golden model 對 RTL 開發的重要性

- 本次 Lab 中，我學到在撰寫複雜 RTL 前，先建立 Python golden model 是非常重要的。
 - 一開始若直接寫 Verilog，很難判斷錯誤發生在哪一個處理階段。

(2) 因此我先使用 Lab7.ipynb 將 Input、Gray、Gaussian、Threshold、Mask、CCL、BBox、Overlay 與 Compare 等流程逐一視覺化。

(3) 透過每一階段的中間圖片，我可以先確認演算法方向是否正確，再依照 Python 結果回推 RTL 架構。

2. Python 也成為後續 Verilog 除錯時的重要參考依據。

(1) 若 RTL 的 gray image 錯誤，可以回頭檢查 RGB channel 擷取與 rounding。

(2) 若 Gaussian 結果不一致，可以檢查 3×3 window、kernel 權重與邊界處理。

(3) 若 mask 結果錯誤，可以檢查 Otsu threshold 與比較條件。

(4) 若 CCL label 或 bounding box 錯誤，可以根據 Python labels 與 overlay image 判斷錯誤來源。

(5) 這讓我學到，大型硬體系統開發不能只靠最後結果除錯，而應該建立中間結果作為 reference。

(三) 對記憶體介面與時序控制的學習

1. 本次作業讓我更熟悉 ImgROM、AnsSRAM 與 UrSRAM 的讀寫控制。

(1) ImgROM 透過 Img_CEN 與 Img_A 讀取 32-bit RGB pixel。

(2) AnsSRAM 與 UrSRAM 則需要同時控制 CEN、WEN、address 與 data。

(3) 特別是 WEN 的定義為 low write、high read，若控制錯誤，資料就不會正確寫入或讀出。

2. 我也學到 memory-based image processing 必須非常注意 address mapping。

(1) 本作業影像大小為 256×256 ，因此總共有 65536 個 pixel。

(2) 位址採 row-major 方式排列，因此 address、row 與 column 之間的對應必須正確。

(3) 若 x、y 或 address 對應錯誤，bounding box 很容易產生一格偏移或整體位置錯誤。

(四) 對 CCL 與 Bounding Box 的理解

1. 在 CCL 部分，我學到 connected component labeling 是整個作業中最關鍵的部分之一。

(1) Lab7 使用 8-connected CCL，因此不能只檢查 N 與 W。

(2) Pass1 需要考慮 NW、N、NE、W 四個已掃描過的鄰居。

(3) 當不同 label 其實屬於同一物件時，需要透過 equivalence 或 union-find 的概念進行合併。

2. Bounding box extraction 讓我理解物件定位的基本方式。

(1) 對每個 label 追蹤 Xmin、Xmax、Ymin、Ymax。

(2) 只要能正確標記每個 connected component，就能從 label 座標中得到物件範圍。

(3) 但如果 CCL label 合併錯誤，bounding box 也會跟著錯誤。

(4) 因此 CCL 與 bounding box 並不是獨立的兩件事，而是前後相依的設計流程。

(五) 對 pixel-level correctness 的體會

1. 本次 Lab 讓我深刻體會到影像處理硬體驗證的嚴格性。

(1) 即使輸出的 bounding box 看起來大致正確，只要有一個 pixel 與 golden 不同，testbench 仍可能判定為錯誤。

(2) 這代表硬體設計不能只追求視覺上相似，而必須達到 pixel-level correctness。

(3) 因此 gray rounding、Gaussian boundary handling、threshold 比較條件、bbox 邊界是否包含端點等細節都非常重要。

2. 我也學到 diff image 對除錯非常有幫助。

(1) 如果 diff image 幾乎全黑，代表輸出與 golden 高度一致。

(2) 如果 diff 出現在 bounding box 邊界，通常代表 overlay 或座標判斷有問題。

(3) 如果 diff 出現在物件區域，可能是 mask、CCL 或 polarity correction 錯誤。

(4) 如果 diff 大範圍出現，則可能是記憶體讀寫或 RGB packing 發生錯誤。

(六) 對 synthesis 與硬體最佳化的學習

1. 本次作業也讓我理解 RTL 正確不代表 synthesis 一定順利。

(1) 有些寫法在 RTL simulation 中可以運作，但合成時可能造成面積過大或編譯時間過久。

(2) 例如使用過大的 full-image register array、過寬的 label memory、過多 nested loop，或過大的 combinational block，都可能讓 synthesis 變得非常慢。

(3) 因此硬體設計必須同時考慮功能正確性與可合成性。

2. 我學到可以透過多種方式改善 synthesis 問題。

(1) 將複雜流程拆成 FSM 多個 state。

(2) 使用暫存器切開過長的 datapath。

(3) 使用 resource sharing 減少重複硬體。

(4) 使用位移取代固定除法，例如 Gaussian filter 的除以 16 可用 right shift 實作。

(5) 避免在單一 combinational always block 中描述過大的運算邏輯。

(6) 在符合功能需求的情況下，降低不必要的暫存器與 memory bit width。

(七) 對驗證流程與工程習慣的學習

1. 本次 Lab 讓我學到完整驗證流程的重要性。

(1) 先用 Python 建立演算法 reference。

(2) 再撰寫 Verilog RTL。

(3) 接著執行 RTL simulation。

(4) 然後進行 SuperLint 檢查。

- (5) 再執行 synthesis。
- (6) 最後進行 post-synthesis simulation。
2. 這個流程讓我更理解 VLSI CAD 作業不是只寫出可以跑的程式，而是要完成可驗證、可合成、可說明、可交付的硬體設計。
 - (1) RTL simulation 用來確認功能正確。
 - (2) SuperLint 用來檢查程式品質與潛在問題。
 - (3) Synthesis 用來確認硬體可實作。
 - (4) Gate-level simulation 用來確認合成後仍能符合功能。
 - (5) Report 則用來清楚說明自己的設計架構、FSM、最佳化方法與驗證結果。

(八) 總結

1. 總結而言，本次 Lab7 讓我從影像處理、RTL 設計、記憶體控制、FSM 架構、CCL 演算法、bounding box extraction、synthesis optimization 到驗證流程，都有更完整的理解。
 - (1) 我學到如何將影像處理演算法拆解成硬體可實作的多個階段。
 - (2) 我學到 Python golden model 對硬體設計與除錯的重要性。
 - (3) 我學到 CCL 與 bounding box 需要精確處理 label、座標與邊界條件。
 - (4) 我學到 RTL 正確之外，還必須考慮 synthesis time、area、timing 與 SuperLint coverage。
 - (5) 我也更加理解 pixel-level correctness 在影像處理硬體驗證中的重要性。
2. 透過這次作業，我不只是完成一個 Draw Bounding Box Engine，也更熟悉從演算法、RTL、模擬、合成到報告整理的完整硬體設計流程。這些經驗對未來進一步學習數位 IC 設計、影像處理硬體加速與 VLSI CAD 工具流程都有很大的幫助。

Please compress all the following files into one compressed file (".tar " format) and submit through Moodle website:

※ NOTE:

1. If there are other files used in your design, please attach the files too and make sure they're properly included.
2. Simulation command

Situation	Command
RTL Simulation	<code>make <u>rtl</u>%</code>
Synthesize your RTL code	<code>make <u>synthesize</u></code>
Post-Synthesis Simulation	<code>make <u>syn</u>%</code>
View Waveform	<code>make <u>nWave</u></code>
Open <u>Superlint</u>	<code>make <u>superlint</u></code>
Delete built files for simulation, synthesis or verification	<code>make <u>clean</u></code>

1. Replace % with 1 ~ 4, to simulate pattern 1 ~ 4 respectively (e.g. `make rtl1`, `make syn2`)

2. There will be a log file called `terminal_result.log` which contains exact same thing of the terminal output, you can clean it with `make clean`